CausalTrace: Runtime Container Defense via Kernel Invariants and Sheaf Coupling

A report submitted in partial fulfilment of the requirements

for the award of the degree of

B.Tech Computer Science and Engineering

by

Shubhankar Bhattacharya
(Roll No: 122CS0047)

Anmol Kashyap
(Roll No: 122CS0039)

Department of Computer Science and Engineering

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

DESIGN AND MANUFACTURING KURNOOL

April 2026

Evaluation Sheet

Title of the Project:

CausalTrace: Runtime Container Defense via Kernel Invariants and Sheaf Coupling

Name of the Student:

- Shubhankar Bhattacharya (Roll No: 122CS0047)
- Anmol Kashyap (Roll No: 122CS0039)

Examiner(s):

_____

Supervisor/Advisor:

_____

Head of the Department:

_____

Date: _____

Place: _____

Declaration

We, the undersigned students, hereby declare that the material presented in the Project Report titled *"CausalTrace: Runtime Container Defense via Kernel Invariants and Sheaf Coupling"* represents original work carried out by us in the Department of Computer Science and Engineering at the Indian Institute of Information Technology, Design and Manufacturing, Kurnool during the academic year **2025–2026**. With our signatures, we certify that:

- We have not manipulated any of the data or results.
- We have not committed any plagiarism of intellectual property.
- We have clearly indicated and referenced the contributions of others.
- We have explicitly acknowledged all collaborative research and discussions.
- We understand that any false claim will result in severe disciplinary action.
- We understand that the work may be screened for any form of academic misconduct.

Shubhankar Bhattacharya (122CS0047) Signature

Anmol Kashyap (122CS0039) Signature

Date: _____

In my capacity as supervisor of the above-mentioned work, I certify that the work presented in this Report is carried out under my supervision, and is worthy of consideration for the requirements of B.Tech. Project work.

Supervisor's Name: **Dr. Anil Kumar R** Supervisor's Signature:

_____

Abstract

Runtime security for containerised workloads sits between two surfaces: the kernel, which observes every syscall, and the user-space control plane, which observes none of them fast enough to matter. Rule-based eBPF agents such as Falco and Tetragon read single syscalls in isolation and flag payloads; they cannot see the *edges* of a multi-step chain that pivots between containers.

This thesis presents **CausalTrace**, a three-tier runtime defence built to close that gap. Tier 1 attaches a tail-call dispatcher to the raw `sys_enter` tracepoint and drives six in-kernel invariant handlers—fork acceleration, execve, sensitive-file, privilege escalation, a `dup2` file-descriptor-type check, and namespace transition. A compound gate routes each breach through a strict, self-inflicted, or trust-gated case, and burns the attacker's IP into a TC classifier that drops their packets at the veth. Tier 2 stitches accepted TCP flows to their remote peer and promotes trust on an L4-stability test. Tier 3 projects each container's syscall bigram into a 74-dimensional signal, applies canonical-correlation restriction maps at rank $k = 15$, and computes Mahalanobis edge energies plus a global Rayleigh quotient over a cellular sheaf Laplacian. Four theorems tie the approach to the system: invariant necessity, bigram-noise adversarial closure, two-layer completeness, and trust-promotion soundness.

Evaluation replays a fixed-seed permutation of 155 attack injections (150 in-distribution drawn from eleven classes, 5 held-out `memfd_create` fileless payloads) against every detector under an identical workload on a 22-container mesh. CausalTrace catches 154 of the 155 injections (99.4 % [96, 100]); Falco tuned reaches 70.3 % [63, 77] and Tetragon tuned reaches 1.3 % [0, 5]. Cross-container lateral movement, chained SSRF → RCE, and fileless out-of-distribution payloads are detected at 100 % while every rule-based baseline misses them entirely. Chapter 6 reports the full per-scenario breakdown with Wilson 95 % confidence intervals.

Acknowledgements

We extend our sincere thanks to our supervisor Dr. Anil Kumar R, Assistant Professor (Grade-I), Department of Computer Science and Engineering, IIITDM Kurnool, for his guidance, patience, and the regular reviews that kept this project on course through its full length. His feedback at every stage improved both the technical direction of the work and the clarity of its presentation.

We also thank the Department of Computer Science and Engineering and the Indian Institute of Information Technology, Design and Manufacturing, Kurnool, for providing the academic environment in which this project was undertaken. We are thankful to our families for their constant support, and to our peers for the many discussions that shaped this work along the way. Any errors that remain are ours.

Contents

Evaluation Sheet i

Declaration ii

Abstract iii

Acknowledgements iv

Contents v

List of Figures ix

List of Tables x

Abbreviations xi

Symbols xii

1 Introduction 1
- 1.1 The shared-kernel attack surface . . . 1
- 1.2 Why per-container detectors are blind to composite attacks . . . 2
- 1.3 Problem statement . . . 3
- 1.4 Contributions . . . 3
  - A three-tier architecture with a strict dependency rule. . . . 4
  - TC-drop session severance. . . . 4
  - A rank-15 CCA calibration that fits real calibration windows. 4
  - A seed-reproducible evaluation harness. . . . 4

2 Literature Review and Research Gaps 5
- 2.1 Production runtime security tools . . . 5
- 2.2 Anomaly-based syscall detectors . . . 6
- 2.3 In-kernel enforcement and adversarial evasion . . . 6

2.4 Cross-container attack research . . . 7
2.5 Mathematical foundations . . . 7
2.6 Capability matrix and research gaps . . . 7
G1 — Semantic blindness of rule-based detectors. . . . 8
G2 — No cross-container chain detection. . . . 8
G3 — Userspace enforcement bottleneck. . . . 8
G4 — Coarse enforcement target. . . . 8
G5 — No mathematical model of inter-container coupling. . . . 9

3 Theoretical Framework **10**
3.1 Notation . . . 10
3.2 Bigram sketch and the 74-dimensional signal . . . 11
Intuition for the shape. . . . 11
3.3 CCA restriction maps per edge . . . 11
Why $k = 15$ and not $k = 50$. . . . 12
3.4 Mahalanobis edge energy . . . 12
3.5 Cellular sheaf Laplacian and Rayleigh quotient . . . 13
Intuition. . . . 13
3.6 Guarded exponential moving average . . . 13
3.7 Evasion resistance: four theorems . . . 14
What we do *not* prove. . . . 15
3.8 Math-to-code mapping . . . 15

4 System Architecture **16**
4.1 Threat model . . . 16
Adversary capability. . . . 16
Out of scope. . . . 17
Defender capability. . . . 17
4.2 Three-tier architecture . . . 18
The independence invariant. . . . 19
4.3 Tier 1: the dispatcher and six handlers . . . 20
4.4 The compound enforcement gate . . . 22
4.5 Tier 2: attribution and session severance . . . 23
Attribution. . . . 23
Trust promotion. . . . 24
Session severance at the veth. . . . 25
Why not `sockops` TCP RST? . . . 25
4.6 Tier 3: the sheaf daemon . . . 25
4.7 Trust model . . . 27
4.8 Testbed topology . . . 27
4.9 Calibration workflow . . . 28

5 Implementation **30**
5.1 The verifier fights . . . 31
Bug 1: direct `pt_regs` dereference past offset 104. . . . 31

Bug 2: Clang fuses bounded zero-loops into `memset`. 31
Bug 3: unbounded `strncpy`-style loops. 31
Other constraints. 31
5.2 BPF maps and handler attachment 32
5.3 Tier-3 daemon and graduated enforcement 33
5.4 Marathon orchestrator 33

6 Evaluation **35**
6.1 Evaluation protocol 35
Hardware. 35
Attack sequence. 35
Statistical machinery. 36
Detection window. 36
6.2 Calibration output 36
6.3 Per-scenario detection rates 36
CausalTrace holds every scenario. 37
Falco collapses on chain and cross-container attacks. 38
Tetragon's stock policies detect almost nothing. 38
Why the threshold does not trip benign traffic. 38
6.4 Tier-1 enforcement latency 39
6.5 False-positive rate on steady-state traffic 39
6.6 Out-of-distribution evaluation (S11) 40
6.7 Tier attribution and runtime overhead 41
6.8 Threats to validity 42

7 Conclusion **44**
7.1 Summary 44
7.2 Limitations 45
7.3 Future work 45
Multi-host sheaf. 45
Incremental online calibration. 45
IPv6 and accept-thread attribution. 45
Hardware offload for the TC classifier. 46
Formal verification of the compound gate. 46
7.4 Closing remarks 46

A Attack Scenarios **47**

B Key BPF Map Layouts **49**

C Reproduction Recipe **52**

Bibliography **54**

List of Figures

1.1 Thesis scope boundary 3

4.1 Three-tier overview 17
4.2 Normal-traffic data flow 18
4.3 Attack data flow 19
4.4 Threat-model scope 20
4.5 Tier-1 internals 22
4.6 Compound enforcement gate 24
4.7 Tier-3 detection cycle 26
4.8 Per-client trust FSM 27
4.9 Testbed topology 28

6.1 Rayleigh energy over the attack replay 37
6.2 Rayleigh separation on log-x 38
6.3 Detection-latency CDF 40
6.4 Steady-state false-positive rate 41
6.5 Tier breakdown per scenario 42
6.6 Runtime overhead 43

List of Tables

2.1 Capability matrix of reviewed runtime security systems 8

3.1 Math-to-code mapping 15

4.1 Per-scenario invariants 24

5.1 Source-tree file manifest 30
5.2 Key BPF maps 32
5.3 Handler attachment 32

6.1 Per-scenario detection rates 37
6.2 S11 OOD detection 41

A.1 Attack catalogue 47

B.1 Full BPF map schema 49

Abbreviations

BCC	**B**PF **C**ompiler **C**ollection
BPF	**B**erkeley **P**acket **F**ilter
CCA	**C**anonical **C**orrelation **A**nalysis
CMS	**C**ount-**M**in **S**ketch
eBPF	**e**xtended **B**erkeley **P**acket **F**ilter
FPR	**F**alse **P**ositive **R**ate
kprobe	**K**ernel **Probe**
LSM	**L**inux **S**ecurity **M**odule
LRU	**L**east **R**ecently **U**sed
PCA	**P**rincipal **C**omponent **A**nalysis
OOD	**O**ut-**o**f-**D**istribution
TC	**T**raffic **C**ontrol (Linux)
TID	**T**hread **ID**
TPR	**T**rue **P**ositive **R**ate

Symbols

L_F	cellular sheaf Laplacian over the container graph
$R(x)$	global Rayleigh quotient $x^\top L_F x \,/\, x^\top x$
τ	per-edge Mahalanobis threshold (mean $+\,4\sigma$)
τ_{global}	calibrated Rayleigh threshold
$d = 74$	sheaf signal dimension per container
$k = 15$	CCA restriction-map rank
$\alpha = 0.02$	EMA smoothing factor
B, Δ	bytes-budget and duration gate for trust promotion

Chapter 1

Introduction

A production Kubernetes node shares a single Linux kernel among tens to hundreds of containers. That kernel is a shared fate [1]: one race in namespace teardown, one off-by-one in `copy_from_user`, one exploitable path in `io_uring`—any of these hands every co-tenant on the node to the attacker at once. Container isolation is a policy, not a hardware boundary. It holds until it does not.

This thesis is about the attacks that never need to break that boundary. They walk between containers through syscalls the kernel is already allowed to serve. Each hop reads as benign under any local per-container policy, yet the composition is a credential exfiltration, a lateral pivot, a container escape. The work reported here builds a three-tier runtime defence that treats such compositions as first-class: not deviations from a single container's profile, but *edges* in a graph that the kernel can instrument directly.

1.1 The shared-kernel attack surface

Modern microservice deployments layer an externally reachable tier (a web front-end, an API gateway) over an internal tier (databases, caches, message queues) that the outer tier reaches over a flat overlay network. Both layers share the same kernel, the same cgroup hierarchy, and the same BPF map space.

Consider a concrete chain on the evaluation testbed: `ct_attacker` exploits a server-side request forgery in `ct-nginx`, which proxies a crafted request to `ct-api-gw` on port 8080. The gateway, processing what it reads as a legitimate upstream call, queries `ct-postgres` on port 5432. Each individual syscall along the way—`connect(2)`,

`sendmsg(2)`, `recvmsg(2)`—is permitted by any reasonable per-container policy. The anomaly lives entirely in the *edge* between containers, not in any single container's syscall trace.

Existing open-source defences miss this class of attack by design. Falco [2] matches per-container syscall sequences against a rule library; it fires on payloads. Tetragon [3] attaches kprobes and enforces per-pod network policies. Tracee [4] builds per-process behaviour profiles. None of the three model the relationship *between* containers.

1.2 Why per-container detectors are blind to composite attacks

The dominant IDS paradigm for containers is a variant of Bag-of-System-Calls (BoSC) [5, 6]: build a frequency or sequence profile of the syscalls a container issues under normal operation, then flag deviations at runtime. These detectors are lightweight and interpretable. Their weakness is semantic. They observe *what* a container does, not *why* or *in response to whom*.

We showed this empirically during mid-review. The same `openat("/etc/passwd")` is benign when `apt-get` resolves a user-ID and malicious when a compromised worker reads credentials. The same `clone(2)` burst is benign when `gpgv` verifies package signatures and malicious during a fork bomb. The same `setuid(0)` is required by `runc` during container init and forbidden during request handling [7]. Every benign case forces a whitelist entry into any rule-based detector. Every whitelist entry becomes an evasion vector.

The deeper failure mode is multi-hop composition. Suppose an attacker chain runs: shell spawn in `ct-webapp-a`, lateral connect to `ct-api-gw` on an uncalibrated port, further connect to `ct-postgres` [8]. Each container produces a locally plausible trace. The attack is encoded in the *joint* behaviour of the trio—in the fact that the shell-spawn and the lateral connect occurred inside the same five-second window in the same cgroup, and that the downstream container then opened a connection it had never opened during calibration. Standard BoSC tooling cannot see that joint behaviour; it lacks the mathematical object—a global model of how containers are *supposed to* couple—that would encode it.

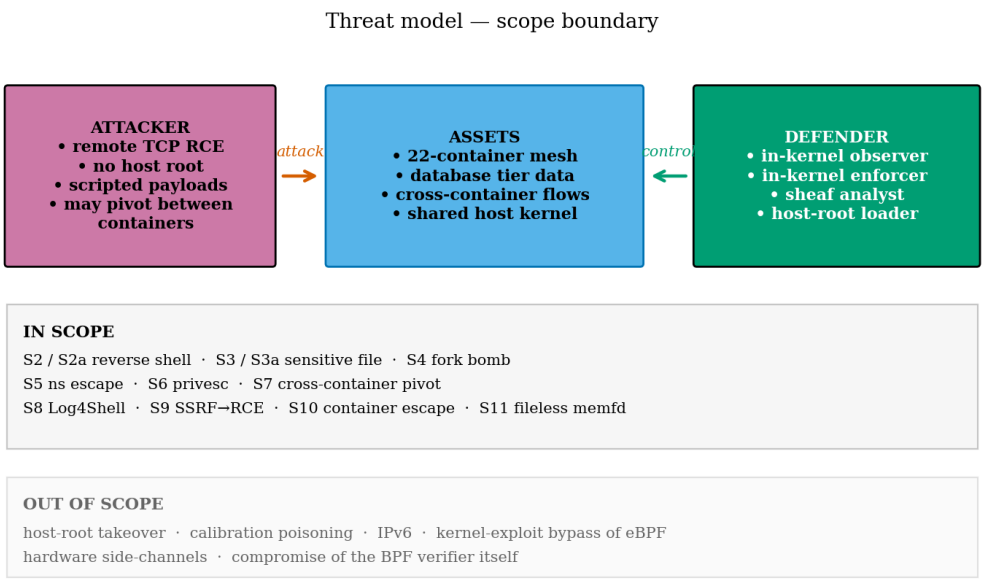

FIGURE 1.1: Scope of the defence. Every attack in the in-scope band is either a physical kernel invariant (caught by Tier 1) or a cross-container edge anomaly (caught by Tier 3). The out-of-scope band lists assumptions that are standard for this class of system.

1.3 Problem statement

Problem. Design, implement, and evaluate a container security system that

1. enforces physical syscall invariants at sub-microsecond latency with no user-space decision in the kill path,
2. attributes every accepted network byte to the client IP that caused it, so enforcement is *surgical*—one attacker IP is severed rather than the entire container process being killed,
3. detects novel cross-container attack chains that no per-container IDS can see, using a principled mathematical model of inter-container coupling, and
4. sustains a realistic concurrent workload with measurable, bounded overhead, producing detection rates with tight confidence intervals against Falco and Tetragon baselines on the same host.

1.4 Contributions

This thesis makes four contributions.

A three-tier architecture with a strict dependency rule. Tier 1 (kernel, 6 eBPF handlers) enforces kernel invariants inline on `sys_enter`. Tier 2 (kprobe on `inet_csk_accept` plus a TC classifier) attributes every accepted byte to a client IP and severs the attacker session at the veth by dropping their packets on an expiring BPF map. Tier 3 (Python daemon) computes a 74-dimensional signal per container per five-second window and passes it through a cellular sheaf Laplacian. The dependency rule is asymmetric: Tier 1 never blocks on Tier 3.

TC-drop session severance. Earlier prototypes called `bpf_send_signal(9)` against the container process, which terminated every live connection the container was serving. We replace this with a `drop_ip_list` BPF map (key: IPv4 source, value: `CLOCK_MONOTONIC` expiry) read by the TC ingress and egress classifiers on each veth. Only packets to or from the burned IP are dropped; other clients are untouched. A five-minute TTL self-heals false positives without operator intervention.

A rank-15 CCA calibration that fits real calibration windows. An earlier iteration of the pipeline fitted CCA at $k = 50$ and produced rank-deficient restriction maps whenever a calibration window contained fewer than sixty aligned samples per edge. We show that $k = 15$ keeps every edge well-conditioned at realistic calibration durations on our 22-container testbed and eliminates the false-positive storm that a rank-deficient covariance inversion causes at runtime.

A seed-reproducible evaluation harness. A fixed random permutation of 150 in-distribution attack injections (drawn from eleven classes) plus five held-out `memfd_create` out-of- distribution injections is replayed identically against CausalTrace, Falco (stock and tuned), and Tetragon (stock and tuned) under the same concurrent background load. The harness produces per-scenario Wilson 95 % confidence intervals [9], detection-latency cumulative distributions, and per-syscall overhead. Chapter 6 reports the results.

Chapter 2

Literature Review and Research Gaps

Prior work on container runtime security falls into four groups: production rule engines, anomaly-based syscall detectors, in-kernel enforcers, and cross-container research. Each group solves a real problem, and each leaves a specific gap that CausalTrace is engineered to close. We frame the review around those gaps.

2.1 Production runtime security tools

Three open-source tools dominate the production market. Falco [2] matches kernel events against a YAML rule library through a modern-eBPF probe. Tetragon [3], a sub-project of Cilium [10], compiles *TracingPolicies* to eBPF kprobes and supports in-kernel enforcement through signal delivery and socket override. Tracee [4] focuses on forensics: it emits events for post-hoc analysis rather than enforcing in the hot path.

All three operate on per-container streams. Rules match inside one container's own syscall trace or against a deny-list; there is no primitive that expresses *container A spawns a shell within five seconds of container B connecting to a previously-unseen port.* A correlation layer can be built on top of any of them, but the round-trip to userspace that the layer forces up wipes out the latency advantage that in-kernel detection was supposed to provide.

Ghimire *et al.* [7] (eBPF-PATROL) are the closest published baseline to our Tier 1 design. They intercept six dangerous syscalls (`clone`, `execve`, `openat`, `setns`, `unshare`, `ptrace`)

and enforce through BPF LSM hooks at ∼100 ns and `bpf_override_return` at ∼200 ns. They miss the same cross-container class that Falco and Tetragon miss, and they require a twelve-entry infrastructure whitelist (documented by them and reproduced in our own mid-review) to keep Docker's own runtime from tripping the rules.

2.2 Anomaly-based syscall detectors

The Bag-of-System-Calls (BoSC) family represents a container's behaviour as a frequency vector over syscall opcodes. Bertinatto *et al.* [5] attach a raw tracepoint to `sys_enter`, filter by mount namespace, and build per-process BoSC vectors. The method cleanly catches one hardcoded container-escape pattern (`openat("/proc/1/ns/mnt")` followed by `setns(CLONE_NEWNS)`) and misses every other class of attack.

Autoencoder-based detectors [6, 11, 12] and sequence models [13, 14] lift this work by learning a density estimate over benign traces. They share the structural weakness: the model learns the *marginal* distribution per container but not the joint structure between containers, so an attacker whose individual syscall frequencies stay close to the marginal can compose an attack whose joint signature is far from any single container's calibrated regime.

2.3 In-kernel enforcement and adversarial evasion

A smaller body of work targets enforcement in kernel space. Zhang *et al.* [15] build a neural-network inference engine that runs inside eBPF through tail calls and integer arithmetic, reporting F1 scores of 0.933 and 0.992 on packet-level intrusion datasets with 5 KB of in-kernel memory. The design is elegant at the verifier boundary, but the detection target is packet flows, not multi-container chains.

Takabayashi and Mambo [16] study the adversarial other side: how an attacker can *evade* eBPF-based detectors by `LD_PRELOAD`-rewriting syscall arguments, hiding process files through `getdents` rewriting, or dropping payload packets through XDP. Their evasion results motivate the strict-invariant design philosophy of CausalTrace's Tier 1: we hook on physical contract violations (a process whose standard input points to a socket is a reverse shell) that the attacker cannot semantically rewrite away.

2.4 Cross-container attack research

He *et al.* [8] document the cross-container attack class. They catalogue five vulnerable online services in which offensive eBPF features break container isolation, and three cloud providers' default Kubernetes clusters vulnerable to cross-node attacks. Their contribution is attack discovery; they propose no defender.

Zhang *et al.* [17] (Pamir-Detector) monitors path-resolution syscalls for filesystem-isolation bypasses (symlink attacks, procfs traversal, host-process-mediated file access). Their coverage is one escape family; CausalTrace's Tier 1 handlers cover complementary families (socket-stdio reverse shells, namespace manipulation, setuid-0 from non-root). No published defender models the joint behaviour of multiple containers mathematically and detects composite attacks that never trip a single-container rule. That is the gap this thesis closes.

2.5 Mathematical foundations

The Tier 3 detector reuses four classical constructions as stated; no new mathematical theorems about them are claimed.

Canonical-correlation analysis [18] supplies the restriction maps F_u, F_v per edge of the communication graph. The Mahalanobis distance [19] scores residuals against the learned coupling covariance. The Count-Min Sketch [20] gives our in-kernel bigram counter at 4×128 per container. Cellular-sheaf spectral theory [21, 22, 23] provides the sheaf Laplacian L_F and the Rayleigh-quotient interpretation as a coupling-consistency measure. The sheaf specialises to the ordinary graph Laplacian of spectral graph theory [24] whenever every restriction map is the identity.

2.6 Capability matrix and research gaps

Table 2.1 summarises where each reviewed system sits against the capabilities we consider essential for defending a composite cross-container attack. A single row is filled in by CausalTrace.

From this matrix and the preceding review we distil five research gaps.

TABLE 2.1: Capabilities of reviewed runtime security systems. ✓ = full, ∼ = partial, ✗ = absent. †In-kernel enforcement is possible via `bpf_send_signal` but is community-discouraged in production because it terminates the entire container process.

System	Cross-container	Kernel-native enf.	Sub-μs latency	OOD detect.	No labelled train data	Attacker-IP severance
Falco [2]	✗	✗†	✗	✗	✓	✗
Tetragon [3]	✗	∼	✓	✗	✓	✗
Tracee [4]	✗	✗	✗	✗	✓	✗
BoSC [5]	✗	✗	✗	✗	✗	✗
eBPF-PATROL [7]	✗	✓	✓	✗	✓	✗
LSTM / LM [13, 14]	✗	✗	✗	∼	✗	✗
Pamir-Detector [17]	✗	✓	✓	✗	✓	✗
CausalTrace (this work)	✓	✓	✓	✓	✓	✓

G1 — Semantic blindness of rule-based detectors. Baseline B (our mid-review Modified PATROL) detected all five attack classes but required twelve infrastructure whitelist entries (`apt-get`, `gpgv`, `runc`, `containerd`, and eight others) to keep normal Docker operations from tripping the rules. Each entry is both a maintenance burden and an evasion vector. The gap is semantic: identical syscalls have different intent depending on context, and no amount of rule tuning closes a gap that lives in meaning rather than syntax.

G2 — No cross-container chain detection. Every reviewed defender operates on per-container events. Multi-stage chains—Log4Shell in a webapp, Dirty-Pipe for privilege escalation, a cgroup escape to the host—are invisible to a single-event monitor because each stage is locally plausible. He *et al.* [8] document the attack class and propose no defender.

G3 — Userspace enforcement bottleneck. PATROL-class architectures route every decision through a userspace Go controller: eBPF probe → ring buffer → controller → `kill()`. Mid-review measurements on that path recorded approximately 23 μs round-trip; the triggering syscall completes before the kill arrives.

G4 — Coarse enforcement target. Kernel-native enforcement delivered through `bpf_send_signal(SIGKILL)` targets the container process itself. In a deployment serving legitimate users alongside the attacker, this is collateral damage—every in-flight request terminates the moment an invariant fires. No reviewed system severs at attacker-IP granularity.

G5 — No mathematical model of inter-container coupling. The object that encodes *which containers may communicate, at what cadence, and with what joint syscall distribution* is absent from every reviewed system. BoSC operates on per-container marginals; network detectors operate on per-flow features; neither captures the joint structure that a sheaf Laplacian makes explicit.

Chapters 3–6 close these five gaps, respectively through the math, the architecture, the implementation, and the evaluation.

Chapter 3

Theoretical Framework

The Tier 3 detector runs a small mathematical pipeline every detection cycle: a bigram sketch becomes a 74-dimensional signal, the signal is projected into an edge-local coupling subspace by a CCA-learned restriction map, Mahalanobis energy scores the edge, and a global Rayleigh quotient over a cellular sheaf Laplacian scores the whole container graph. A guarded exponential moving average keeps the global threshold honest under normal drift without letting a slow-drip attack poison it.

We state the classical pieces as given and cite primary sources [18, 19, 21, 22, 23, 24]. No new mathematics about these objects is claimed. The contribution of this chapter is the assembly into a deployable detector and the four theorems in Section 3.7, which capture why that assembly is hard to evade.

3.1 Notation

$C = \{c_1, \ldots, c_n\}$ is the set of monitored containers, each with cgroup identifier $g_i \in \mathbb{N}$. A detection cycle is a window of length T indexed by t. At cycle t each container produces a signal $\mathbf{x}_i^{(t)} \in \mathbb{R}^d$ with $d = 74$. The communication graph $G = (V, E)$ has $V = C$ and E the set of observed inter-container edges; we write $e = (u, v, \ell)$ with $\ell \in \{0, 1, 2\}$ encoding the temporal lag. After calibration every edge carries restriction maps $F_u^{(e)}, F_v^{(e)} \in \mathbb{R}^{k \times d}$ with $k = 15$, a regularised covariance $\Sigma_e \in \mathbb{R}^{k \times k}$, and a Mahalanobis threshold τ_e. The global Rayleigh threshold is τ_{global}.

3.2 Bigram sketch and the 74-dimensional signal

Let Σ be the set of top-25 syscall opcodes observed across the testbed during calibration (full list in Appendix B). A *bigram* is an ordered pair $(s_i, s_j) \in \Sigma \times \Sigma$ recording that opcode s_j immediately followed s_i in the same cgroup. In-kernel counting uses a Count-Min Sketch [20] with r=4 rows and w=128 columns; the estimate of the bigram count is $\min_\rho \mathrm{CMS}[\rho][h_\rho(\cdot) \bmod w]$. The sketch resets every T seconds, so each cycle observes only current traffic.

The reconstructed 625-dimensional bigram vector $\mathbf{b}^{(t)}$ is compressed into $\mathbf{x}^{(t)} = \phi(\mathbf{b}^{(t)}) \in \mathbb{R}^{74}$ through four blocks:

- **Shannon entropy** (3 dims): marginal entropy of first opcodes, marginal entropy of second opcodes, joint bigram entropy.
- **PCA** (50 dims): projection onto the top-50 principal components of the calibration bigram set, learned once and frozen.
- **Top-k marginals** (20 dims): marginal frequencies of the 20 most common single syscalls.
- **Rate** (1 dim): $\log_{10}$ of total syscalls in the window.

The signal is then whitened per container, $\tilde{\mathbf{x}}_i^{(t)} = (\mathbf{x}_i^{(t)} - \boldsymbol{\mu}_i) \oslash (\mathbf{s}_i + \epsilon)$, with $\epsilon = 10^{-6}$. Diagonal whitening is deliberate: a full 74×74 covariance inversion would be rank-bounded by the per-container sample count during calibration.

Intuition for the shape. The entropy block captures *disorder* in the syscall stream. The PCA block captures *structural pattern* in the bigram sketch. The marginals capture *what* the container is doing. The rate captures *how much* it is doing. A reverse shell perturbs all four blocks; a fork bomb perturbs the rate and entropy blocks; a cross-container pivot perturbs primarily the PCA block. Separating these blocks is what lets the sheaf Laplacian find an attack that would be invisible to any single dimension.

3.3 CCA restriction maps per edge

Canonical-correlation analysis [18] finds, for aligned matrices $X \in \mathbb{R}^{T\times d}$ and $Y \in \mathbb{R}^{T\times d}$, linear projections $F_X, F_Y \in \mathbb{R}^{k\times d}$ that maximise the correlation between $F_X X^\top$ and

$F_Y Y^\top$:

$$(F_X^\star, F_Y^\star) = \arg\max_{F_X, F_Y} \frac{\operatorname{tr}\big(F_X X^\top Y F_Y^\top\big)}{\sqrt{\operatorname{tr}(F_X X^\top X F_X^\top)\ \operatorname{tr}(F_Y Y^\top Y F_Y^\top)}}. \tag{3.1}$$

The solution is a generalised eigendecomposition of the joint covariance matrix; closed forms are in the primary source. For every observed edge $e = (u, v, \ell)$ we form aligned calibration matrices $X_u^{(\ell)} = [\tilde{\mathbf{x}}_u^{(1)}, \ldots, \tilde{\mathbf{x}}_u^{(T-\ell)}]^\top$ and $X_v^{(\ell)} = [\tilde{\mathbf{x}}_v^{(1+\ell)}, \ldots, \tilde{\mathbf{x}}_v^{(T)}]^\top$, and solve (3.1) to obtain $F_u^{(e)}, F_v^{(e)}$. Three lags per edge are calibrated.

Why $k = 15$ and not $k = 50$. The CCA solution is only stable when the per-edge aligned sample count exceeds a rank-dependent floor (roughly $4k$ in our regularised implementation). An early iteration used $k = 50$ and required ≈ 60 aligned samples per edge—a floor that realistic calibration windows could not always meet because inter-container activity alignment is lossy. At $k = 15$ the floor drops to ≈ 25 aligned samples; every edge observed in our testbed calibrates cleanly. Section 6.2 reports that $k = 15$ retains more than 90% of the explained inter-container correlation.

3.4 Mahalanobis edge energy

At runtime cycle t the residual on edge $e = (u, v, \ell)$ is $\mathbf{r}_e^{(t)} = F_u^{(e)} \tilde{\mathbf{x}}_u^{(t)} - F_v^{(e)} \tilde{\mathbf{x}}_v^{(t+\ell)} \in \mathbb{R}^k$. Under the calibrated-coupling null, $\mathbf{r}_e^{(t)}$ is zero-mean Gaussian in the shared subspace. We estimate its covariance with Tikhonov regularisation:

$$\hat{\Sigma}_e = \frac{1}{T-1} \sum_{\tau=1}^{T} \mathbf{r}_e^{(\tau)} \big(\mathbf{r}_e^{(\tau)}\big)^\top + \epsilon_\Sigma I_k, \quad \epsilon_\Sigma = 10^{-6}.$$

The *edge energy* is the Mahalanobis distance [19]:

$$E_e(t) = \big(\mathbf{r}_e^{(t)}\big)^\top \hat{\Sigma}_e^{-1} \mathbf{r}_e^{(t)}. \tag{3.2}$$

Under the null $E_e \sim \chi_k^2$; for k=15 the mean is 15 and the 99% tail is ≈ 30.6. The per-edge threshold is $\tau_e = \hat{\mu}_e + 4\hat{\sigma}_e$.

3.5 Cellular sheaf Laplacian and Rayleigh quotient

A cellular sheaf $\mathcal{F}$ on $G = (V, E)$ assigns a vector space $\mathcal{F}(v) = \mathbb{R}^d$ to each vertex, a vector space $\mathcal{F}(e) = \mathbb{R}^k$ to each edge, and a linear restriction $\mathcal{F}_{v \lhd e} : \mathcal{F}(v) \to \mathcal{F}(e)$ to every incident pair. The coboundary δ acts on an edge (u, v) by $(\delta \mathbf{x})_e = F_u^{(e)} \mathbf{x}_u - F_v^{(e)} \mathbf{x}_v$. The *sheaf Laplacian* is

$$L_{\mathcal{F}} = \delta^\top \delta \in \mathbb{R}^{nd \times nd}. \tag{3.3}$$

The diagonal block for vertex v is $\sum_{e:\, v \lhd e} (F_v^{(e)})^\top F_v^{(e)}$; the off-diagonal block for (u, v) incident at edge e is $-(F_u^{(e)})^\top F_v^{(e)}$. When every restriction is the identity, $L_{\mathcal{F}}$ reduces to the ordinary graph Laplacian [24].

Intuition. Think of each container as a point in its own 74-dimensional configuration space, and each CCA restriction as the pair of projection lenses that edge uses to see its two endpoints. If both endpoints project onto the same point in the shared 15-dimensional subspace, the edge is *happy*. If they disagree, the edge has *tension*. The sheaf Laplacian sums tension across all edges. A low Rayleigh quotient means the graph is at rest; a high one means the coupling is being violated.

The global Rayleigh quotient is

$$R(\mathbf{x}) = \frac{\mathbf{x}^\top L_{\mathcal{F}} \mathbf{x}}{\mathbf{x}^\top \mathbf{x}} = \frac{\sum_{e=(u,v) \in E} \| F_u^{(e)} \mathbf{x}_u - F_v^{(e)} \mathbf{x}_v \|_2^2}{\| \mathbf{x} \|_2^2}. \tag{3.4}$$

We set $\tau_{\text{global}} = \hat{\mu}_R + 4\hat{\sigma}_R$ at calibration and fire the global gate when $R > \tau_{\text{global}}$.

3.6 Guarded exponential moving average

A naive EMA adapts the threshold to current observations but has one fatal mode: a slow-drip attack that never exceeds the instantaneous threshold poisons the average into following the attack. Let $s^{(t)} \in \{0, 1\}$ indicate whether cycle t was pristine (no behaviour bit fired, no edge energy exceeded τ_e, no alert in the ring buffer) and let $p^{(t)} = \sum_{\tau=t-5}^{t} s^{(\tau)}$ be the pristine-streak length over the last six cycles. The guarded update is

$$\tau^{(t)} = \begin{cases} \alpha R^{(t)} + (1 - \alpha) \tau^{(t-1)} & \text{if } p^{(t)} = 6, \\ \tau^{(t-1)} & \text{otherwise.} \end{cases} \tag{3.5}$$

Any suspected anomaly freezes the threshold. We use $\alpha = 0.02$, giving an effective half-life of roughly 34 pristine cycles—slow enough to track legitimate drift, fast enough to recover after a clean recovery window.

3.7 Evasion resistance: four theorems

The design of CausalTrace rests on four defensible claims. Each theorem answers a specific question a reviewer or a panel member would raise. The proofs are informal because each relies on a kernel-observable mechanism rather than a closed-form reduction; the citations point to the kernel or userspace file that realises the mechanism.

Theorem 3.1 (Invariant necessity)**.** *Let $\mathcal{A}_I \subseteq \mathcal{A}$ be the set of attack classes that require the attacker to emit a specific syscall pattern π to succeed (for example, an interactive reverse shell requires* $\mathrm{dup2}(\texttt{sockfd}, k)$ *for some $k \in \{0, 1, 2\}$). For every $\alpha \in \mathcal{A}_I$, a kernel hook observing π achieves recall $= 1$: the attacker cannot complete α without the hook firing. Table 4.1 maps each scenario to its invariant.*

Theorem 3.2 (Bigram-noise adversarial closure)**.** *Let N be the noise-syscall whitelist filtered before the bigram update (Section 3.2). An adversary attempting to dilute the bigram-count vector during an attack faces a dichotomy:*

1. *Emit syscalls in N. These are filtered at the kernel and contribute nothing to the sketch. They cost CPU and nothing else.*
2. *Emit syscalls outside N. Each such syscall produces a bigram* (prev, s) *whose probability under the calibrated model is low (it was not seen during calibration). The resulting $\tilde{\mathbf{x}}^{(t)}$ moves farther from the calibrated mean, the Mahalanobis edge energy* (3.2) *rises, and the sheaf Rayleigh quotient rises with it.*

The attacker has no noise-channel at the bigram layer: every evasion attempt triggers the statistics.

Theorem 3.3 (Two-layer completeness)**.** *Partition the attack space as $\mathcal{A} = \mathcal{A}_I \cup \mathcal{A}_U$, where $\mathcal{A}_U$ is the set of novel attacks without a pre-enumerated kernel invariant. Theorem 3.1 gives Tier 1 recall $= 1$ on $\mathcal{A}_I$. Tier 3 achieves recall $\geq 1 - \beta$ on $\mathcal{A}_U$, where β is controlled by the per-edge false-positive rate $\Pr[E_e > \tau_e \mid null]$ and the 4σ gate in Sections 3.4 and 3.5. The composition covers $\mathcal{A}$ with detection rate bounded below by the invariant coverage on the known set and the sheaf's statistical power on the novel set.*

Theorem 3.4 (Trust-promotion soundness)**.** *Let $B = 5120$ bytes and $\Delta = 1$ second be the L4-stability gate for trust promotion (Chapter 4, Section 4.7). A client IP reaches* CALIBRATED *only after a single TCP flow accumulates at least B bytes over at least Δ seconds. An attacker whose entire session budget is under B bytes cannot promote, and therefore cannot bypass Case A enforcement; a burst attacker who fires and disconnects before Δ elapses is always routed through the untrusted path.*

Remark 3.5 (Unforgeable attribution). The per-TID client-IP map is populated by the `inet_csk_accept` kretprobe using kernel-owned `struct sock` fields. No userspace write path exists to this map; an attacker cannot spoof a client-IP entry without kernel-level privilege escalation, which is outside the threat model of Chapter 4.

What we do *not* prove. The sheaf detector's β in Theorem 3.3 is an empirical quantity—we do not claim a closed-form upper bound. We do not claim that Theorem 3.1 holds for attack classes that a future kernel may allow through new syscalls not yet in Σ. Theorem 3.4 assumes the attacker cannot fabricate TCP traffic on the defender's behalf, which holds as long as the defender's veth is the only way into the container.

3.8 Math-to-code mapping

TABLE 3.1: Where each mathematical object is realised in the codebase.

Object / equation	Kernel side	Userspace side
Bigram CMS, 4×128	`kernel/causaltrace_bcc.c:613`	`tier3/signal_extractor.py`
Signal extraction ϕ, d=74	—	`tier3/signal_extractor.py`
Per-container whitening	—	`tier3/whitener.py`
CCA restriction maps (3.1)	—	`tier3/calibrate.py`
Mahalanobis energy (3.2)	—	`tier3/sheaf_detector.py`
Sheaf Laplacian (3.3)	—	`tier3/sheaf_detector.py`
Rayleigh quotient (3.4)	—	`tier3/sheaf_detector.py`
Guarded EMA (3.5)	—	`tier3/ema_buffer.py`
Invariant hooks (Thm. 3.1)	`kernel/causaltrace_bcc.c`	—
Trust FSM (Thm. 3.4)	`kernel/causaltrace_bcc.c:459`	`tier3/trust_promoter.py`

The per-scenario invariant map (Table 4.1) is deferred to Chapter 4 where the full attack catalogue is introduced.

Chapter 4

System Architecture

Figure 4.1 gives the high-level view. A raw `sys_enter` tracepoint feeds a tail-call dispatcher that routes each opcode to one of six invariant handlers. A compound gate `maybe_kill(cg, case)` serialises every enforcement request. Tier 2 attributes network traffic to client IPs and severs burned sessions at the veth. Tier 3 runs the sheaf detector in userspace every detection cycle and writes verdicts to a JSON log. The three tiers talk to each other only through shared BPF maps, so Tier 1 never blocks on Tier 3.

Figures 4.2 and 4.3 trace the information flow through the three tiers in the two operational regimes we care about: a benign request that completes normally, and an attack that fires a strict invariant and burns the attacker's IP. The numbered steps correspond one-to-one with the implementation paths in Chapter 5.

4.1 Threat model

Figure 4.4 summarises the scope. The primary asset is the confidentiality, integrity, and availability of the monitored containers' workloads and of the database tier's data at rest. Secondary assets are the host's other tenants (no cross-container leakage) and the host itself (no escape to the root namespace).

Adversary capability. The adversary has code execution inside a monitored container, at the privilege level of the container's main process. The adversary can originate this code execution either from outside through a network vulnerability (RCE,

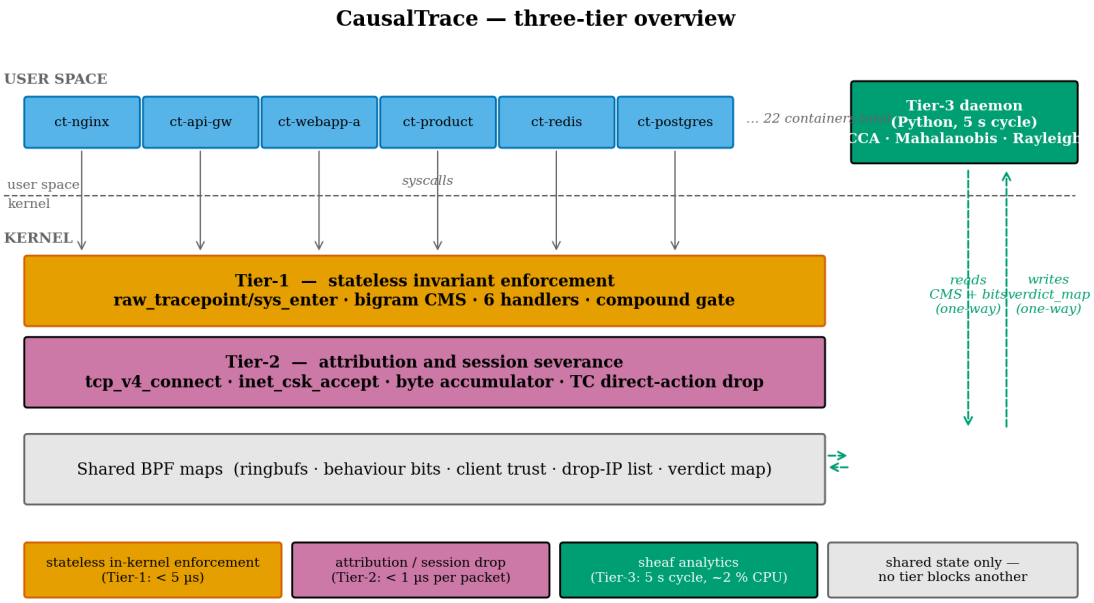

FIGURE 4.1: Three-tier overview. Containers sit above the user–kernel boundary; Tier 1 (stateless invariant enforcement) and Tier 2 (attribution + TC drop) live entirely in the kernel; Tier 3 runs in userspace and reads state through shared BPF maps only. No tier blocks any other tier—if Tier 3 crashes, Tier 1 and Tier 2 keep enforcing.

SSRF, Log4Shell-class) or by pivoting from a peer container that was compromised first. The adversary does not hold host root.

Out of scope. We exclude: attackers with host root (who can unload BPF programs directly); attackers with a kernel exploit that writes kernel memory outside the BPF verifier's model; attackers who subvert the BPF verifier itself; denial-of-service attacks that overwhelm the ring buffers before any alert can escape. These are legitimate attack classes but lie outside what an in-kernel defender can defeat.

Defender capability. We assume a host-root loader that attaches the eBPF programs at boot, a Python daemon with access to BPF maps (running as the same UID as the loader), and network administrator capability (`CAP_NET_ADMIN`) to attach TC filters to container veths.

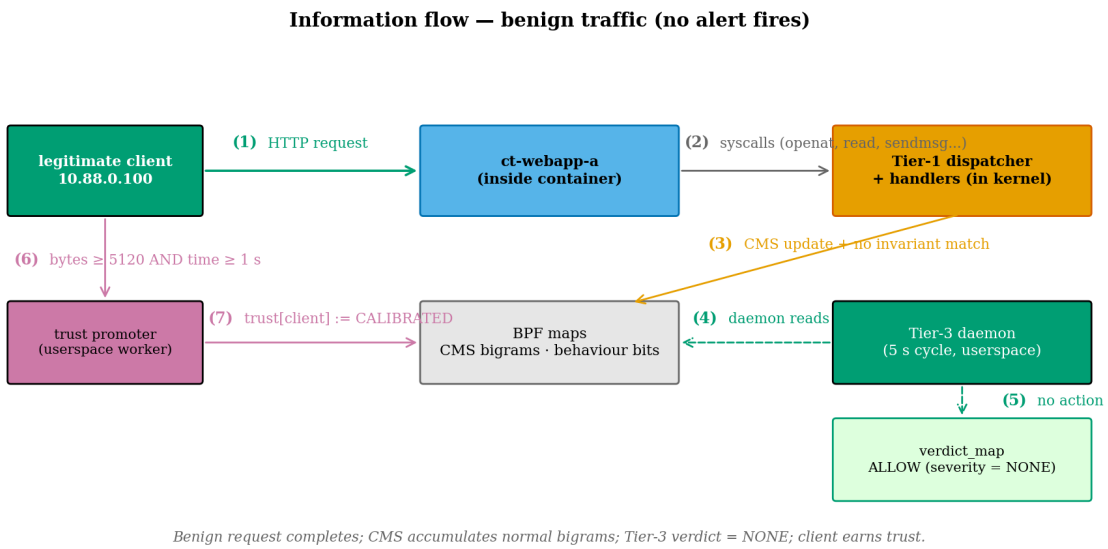

FIGURE 4.2: Information flow on benign traffic. The client issues an HTTP request (1); syscalls reach the Tier-1 dispatcher (2); the CMS is updated but no invariant matches (3); Tier-3 reads the accumulated state on its next cycle (4) and writes an ALLOW verdict (5); meanwhile the trust promoter observes flow stability (6–7) and upgrades the client to CALIBRATED. No kill, no drop.

4.2 Three-tier architecture

We enumerate the three tiers in data-flow order.

Tier 1 (kernel, stateless). A raw tracepoint attached to `sys_enter` sees every syscall issued by every process on the host. A dispatcher filters out host processes via mount-namespace comparison, updates a per-cgroup Count-Min Sketch of the syscall bigram, and tail-calls to one of six specialist handlers based on the syscall opcode. Each handler tests a physical invariant (dup2 socket-to-stdio, shell-exec with socket-fd, setuid(0) from non-root, unshare with CLONE_NEWNS, path-class match on openat, fork acceleration) and either returns without action, updates a behaviour bit in a shared map, or enters the compound-gate decision of Section 4.4.

Tier 2 (kernel, attribution). Two kprobes (`tcp_v4_connect` and `inet_csk_accept`) and two pairs of byte-accounting kprobes (`tcp_sendmsg`/`tcp_recvmsg`) attribute every network byte to a remote peer IP. A per-TID LRU map of 16,384 slots keeps the attribution fresh across fork-exec worker patterns. A per-socket context map of 8,192 slots accumulates byte counts for the L4-stability-based trust promotion described in Section 4.7. When the compound gate decides to sever an attacker's session, Tier 2 writes the attacker IP to

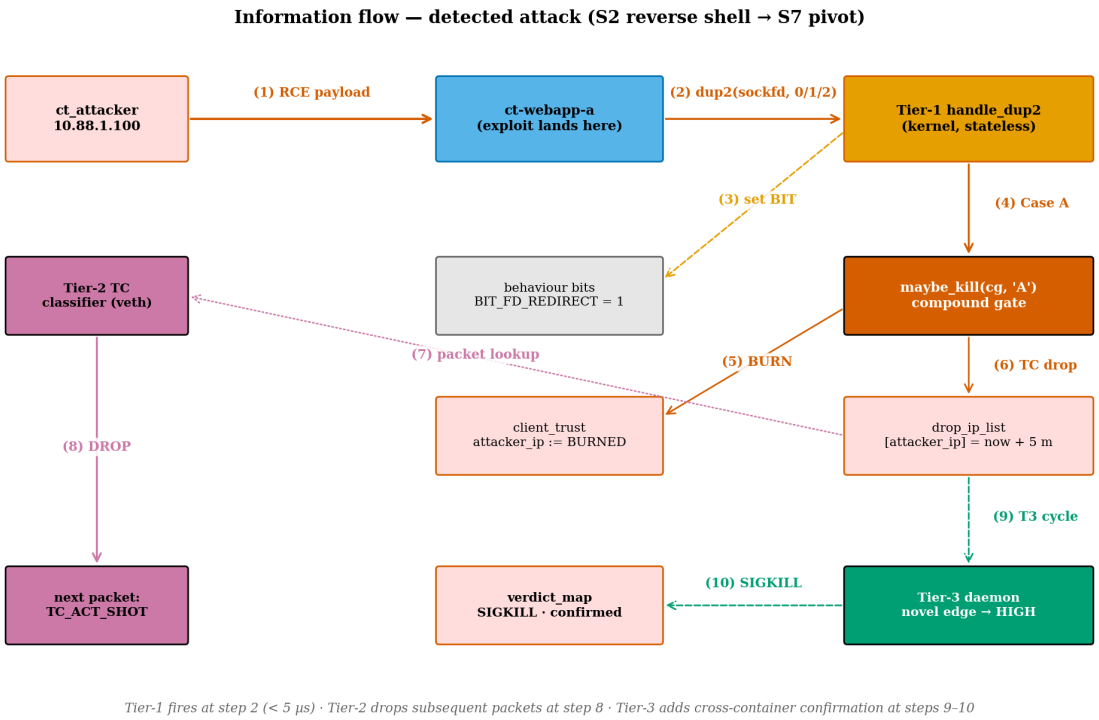

FIGURE 4.3: Information flow on a detected attack. An RCE payload lands in a webapp (1–2); `handle_dup2` fires on the socket-to-stdio `dup2` (3–4); the compound gate writes the attacker IP into `drop_ip_list` and burns the trust entry (5–6); the TC classifier drops all subsequent attacker packets at the veth (7–8); Tier-3's next cycle produces an independent cross-container confirmation (9–10). Tier-1 acts in the sub-microsecond regime; Tier-2 severs the session at the network layer within a packet; Tier-3 adds graph-level evidence on the 5-second cadence.

`drop_ip_list` with a 5-minute TTL, and the direct-action TC classifier on each container's veth silently returns `TC_ACT_SHOT` for any packet matching the burned IP.

Tier 3 (user-space, analytics). A Python daemon reads the Count-Min Sketches and behaviour bits from the kernel every five seconds, extracts the 74-dimensional signal per container, computes the Mahalanobis edge energies and the global Rayleigh quotient through the mathematics of Chapter 3, and writes verdicts to a JSONL log. The daemon may also write into the enforcement maps (`deny_connect_map`, `deny_openat_map`, `fw_allow_map`, etc.) to graduate enforcement under confirmed anomalies. Tier 3 is optional: the kernel continues to enforce strict invariants regardless of daemon state.

The independence invariant. The three tiers communicate only through BPF maps. Tier 1 never blocks on Tier 3. Tier 3 never blocks Tier 1. If the Python daemon crashes, segfaults, or runs out of memory, Tier 1 and Tier 2 remain attached and enforcing; the

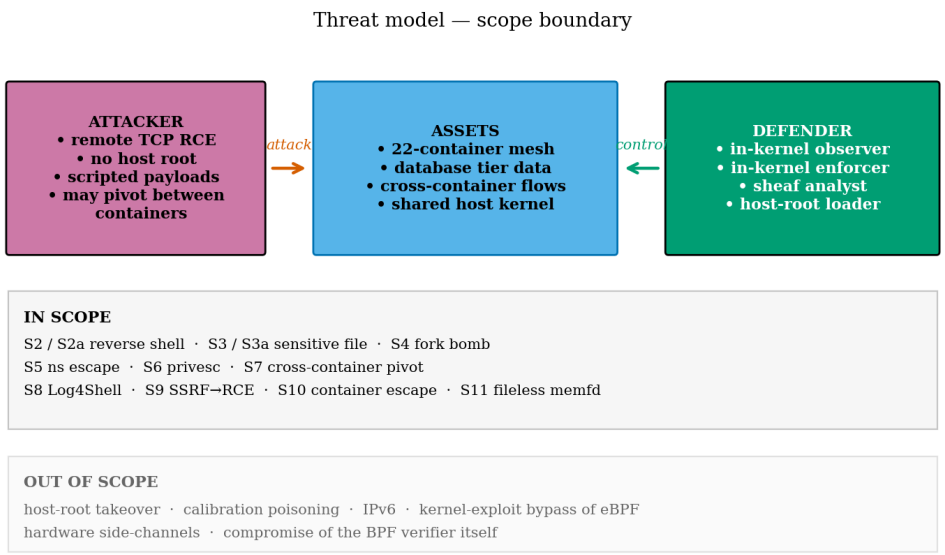

FIGURE 4.4: In-scope attacks (S2 through S11) and explicit out-of-scope assumptions. The defender operates inside the kernel and assumes host root is not compromised.

only degradation is the loss of the sheaf-coupling detection surface. This decoupling is the structural answer to research gap G3 (detection-vs-enforcement split) from Chapter 2.

4.3 Tier 1: the dispatcher and six handlers

The Tier 1 dispatcher runs six steps on every syscall:

1. Read the syscall number from `ctx->args[1]`.
2. Compute the current task's mount namespace inode and compare against the single-slot `host_ns` array. Host-side processes return immediately.
3. Check the cgroup's verdict map for a Tier 3 KILL decree and SIGKILL if present.
4. Resolve pending cgroup inheritance (for post-unshare children).
5. Update the Count-Min bigram sketch (unless this syscall is on the noise list of `getpid`, `gettid`, `getuid`, `getppid`, `time`, `clock_gettime`).
6. Tail-call through `prog_array[syscall_nr]` to the specialist handler.

The tail-call is an unconditional branch; cold paths where no specialist handler is registered for the opcode still fall through to the map lookup and simply return 0 from the program-array call. This keeps the hot-path overhead to a single comparison plus an indirect branch, measured at approximately 65 ns on our evaluation kernel.

The six handlers cover the strict-invariant attack classes we target:

`handle_dup2` inspects `dup2(oldfd, newfd)`. If `newfd` is 0, 1, or 2 and the file pointed to by `oldfd` is a socket (`i_mode & 0xF000 == S_IFSOCK`), the handler sets `BIT_FD_REDIRECT`, emits an alert, and calls `maybe_kill(cg, 'A')`. No legitimate workload redirects stdio onto a socket.

`handle_fork` counts `clone(2)` and `clone3(2)` per-cgroup per-second. The handler computes the second discrete derivative $d^2 = \text{rate}_t - 2\,\text{rate}_{t-1} + \text{rate}_{t-2}$ and fires when $d^2 > 0$ is sustained with monotonic growth, or when the raw rate exceeds 500/s. The second derivative catches the acceleration phase before the absolute rate reaches fork-bomb territory. This is a self-inflicted kill; Case D in the compound gate.

`handle_execve` extracts the basename of the exec'd filename (in-kernel byte-by-byte string comparison because eBPF forbids `strcmp`) and matches against {`sh, bash, dash, nc, ncat, zsh`}. If the binary is a shell, the handler inspects fds 0, 1, 2: if any is a socket, this is a reverse shell staged through shell redirection (`bash -i >& /dev/tcp/host/port`), a strict invariant. Otherwise, plain shell exec is Case X, trust-gated.

`handle_file` (`openat`) and `handle_file_open` (`open`) classify the requested path against a pruned trie: `/etc/sha` prefix → sensitive-credentials read; `/proc/1/` prefix → namespace probe; `/var/run/sec` prefix → Kubernetes secrets. The handler sets the matching bit but does not kill (file-reads alone are ambiguous; we defer to the compound gate with Tier 3 confirmation).

`handle_privesc` dispatches on syscall number: `setuid(105)` → check that target uid is 0 and current uid is non-zero → Case A. `unshare(272)` → check for CLONE_NEWNS or CLONE_NEWUSER flags → Case A. `setns(308)` and `ptrace(101)` → unconditional Case A inside a container.

`trace_connect_return` (not strictly a tail-called handler but part of the Tier 1 enforcement surface). On successful `tcp_v4_connect`, the handler looks up the

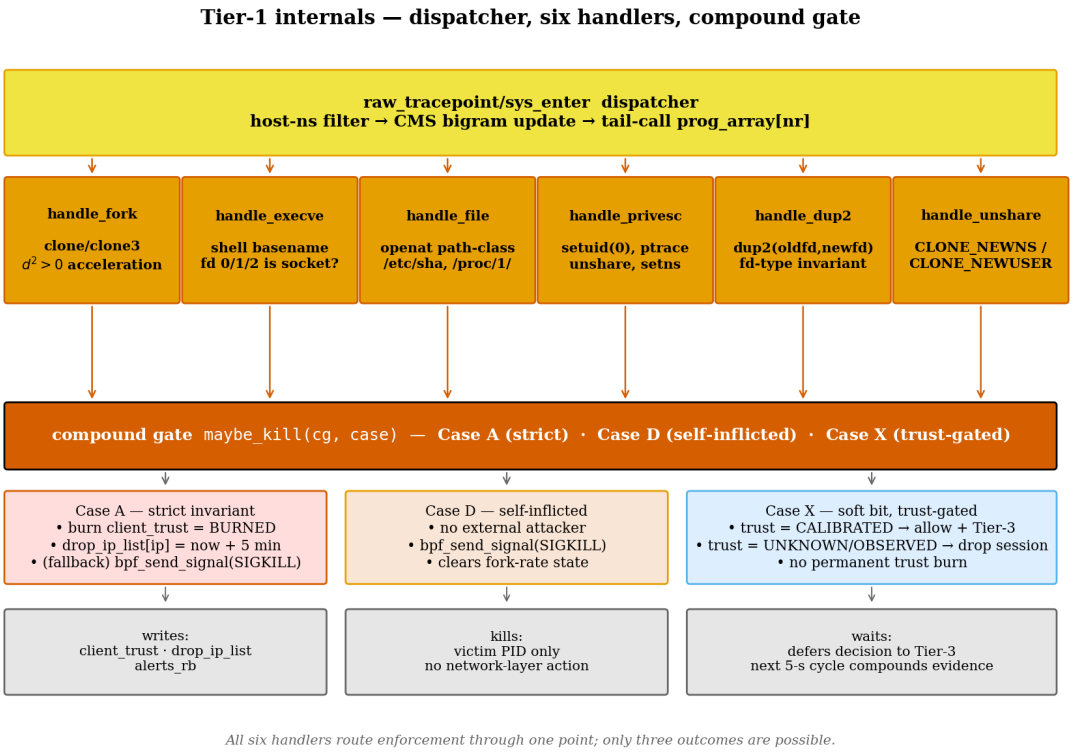

FIGURE 4.5: Tier-1 dispatcher, the six specialist handlers, and the compound gate. The dispatcher performs the host-ns filter and CMS bigram update, then tail-calls the opcode-specific handler. Every handler terminates at `maybe_kill(cg, case)`; the gate routes to exactly one of three outcomes (Case A / D / X).

destination container via `ip_to_cgroup`. If the source cgroup already has `BIT_SHELL_SPAWN` or `BIT_FD_REDIRECT` set within the last 5 s, this is a compound confirmed two-hop attack and the handler calls `maybe_kill(src_cg, 'A')`.

Each handler's hot path completes in the 280–640 ns band on the evaluation host (Intel Core i7-10710U, kernel 6.17, BCC-compiled BPF). Figure 4.5 shows the full dispatcher/handler/gate topology; every handler is a tail-call from the dispatcher and every enforcement request funnels through `maybe_kill`.

4.4 The compound enforcement gate

The compound gate `maybe_kill(cg, case)` is the single kill decision point in the kernel. All six handlers funnel enforcement requests through it. The gate takes a cgroup identifier and a *case tag* that classifies the triggering event:

Case A (Strict invariant). Physical contract violation: dup2 socket-to-stdio, shell exec with socket stdio, setuid(0), unshare with namespace flags, two-hop compound. The gate burns the attacker's client IP (if attributable) into `client_trust` and writes the IP into `drop_ip_list` with a 5-minute TTL. If the client IP is not attributable (some strict invariants have no external attacker), the gate falls back to `bpf_send_signal(SIGKILL)`.

Case D (Self-inflicted). Fork bomb, runaway workload. No external attacker to drop at the veth. The gate sends `SIGKILL` to stop the workload.

Case X (Soft-bit, trust-gated). Plain shell exec, single sensitive-file read, single lateral connect to an uncalibrated port without an accompanying invariant. The gate reads the client IP's trust level. If CALIBRATED, the gate returns without action and relies on Tier 3 to compound-confirm; if UNKNOWN, OBSERVED, or BURNED, the gate treats the event as Case C: write the IP to `drop_ip_list`, but do not burn trust permanently.

This three-case taxonomy reflects a design observation: different categories of evidence merit different enforcement targets. Physical invariants warrant permanent trust burn plus session drop. Self-inflicted events warrant stopping the workload. Soft anomalies from trusted clients warrant waiting for compound confirmation rather than silent collateral damage. Legacy designs that compressed all three into a single `SIGKILL(process)` pathway suffered G4 (coarse enforcement target).

Table 4.1 closes the loop on Theorem 3.1 by mapping each attack scenario to the invariant that catches it.

4.5 Tier 2: attribution and session severance

Tier 2's job is to make the compound gate's decision targetable. Two structures support this.

Attribution. The per-TID LRU map `tid_client_ip` (16,384 slots) records the client IP from which each thread received its most-recent work. The map is populated on `inet_csk_accept`: the accepting thread's pid/tgid is the key, the peer IP extracted from `sock->__sk_common.skc_daddr` is the value. A fork hook (`sched_process_fork` tracepoint) propagates the attribution from parent to child so that forked workers inherit

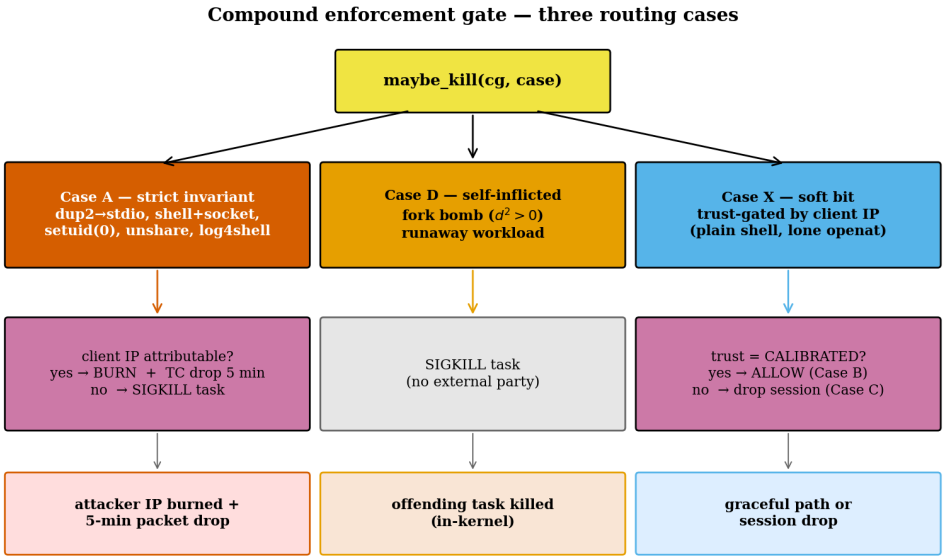

FIGURE 4.6: Three routing cases of `maybe_kill(cg, case)`. Case A burns the attacker's IP and drops all its packets at the veth (SIGKILL is a fallback when no IP is attributable). Case D kills the offending task because no external attacker exists. Case X reads trust state and takes a grace path for calibrated clients or drops the session otherwise.

TABLE 4.1: Attack scenarios and the Tier-1 invariant that catches each. Every row is the necessary-condition claim of Theorem 3.1 realised in a kernel hook.

ID	Scenario	Necessary syscall pattern	Ke
S2	Reverse shell	`dup2(sockfd, 0|1|2)`	ha
S2a	Revshell + noise pad	`dup2(sockfd, 0|1|2)`	ha
S3	`/etc/shadow` read	`openat("/etc/sha*")`	ha
S3a	Sensitive + noise pad	`openat("/etc/sha*")`	ha
S4	Fork bomb	$d^2(\text{rate})/dt^2 > 0$	ha
S5	Namespace escape	`nsenter` → `setns(fd, _NEWNS)`	ha
S6	Privilege escalation	`unshare(_NEWUSER|_NEWNS)` + `setuid(0)`	ha
S7	Cross-container pivot	TIER-1 two-hop + TIER-3 sheaf	tr
S8	Log4Shell chain	`execve(shell)` + socket	ha
S9	SSRF → RCE	TIER-3 sheaf edge spike	sh
S10	Container escape	rare-syscall chain: `unshare`, `mount`, `core_pattern` write, `memfd+exec`, `bpf()`	ha
S11	`memfd_create` OOD	`execveat(/proc/self/fd/N)`	ha

the IP of the request that spawned them. A per-cgroup fallback map `cgroup_current_client` records the most recent accepted IP per cgroup; it serves as a best-effort fallback when per-TID attribution misses (for instance in accept-thread-plus-thread-pool servers).

Trust promotion. The trust promoter (a userspace worker, not in the kernel hot path) reads the per-socket byte-accumulation map `connection_context` every few seconds. A

client IP is promoted from OBSERVED to CALIBRATED when any single flow from that IP accumulates ≥ 5120 bytes and runs for ≥ 1 second (the L4-stability rule). This simple rule distinguishes a client that completes a real HTTP transaction from one that merely opens a TCP connection and disconnects. A client IP is permanently demoted to BURNED when any Case A strict invariant fires on a syscall attributed to that IP.

Session severance at the veth. When the compound gate decides to drop an attacker, it writes the attacker IP to `drop_ip_list` with expiry `bpf_ktime_get_ns() + 5 min`. A TC classifier program, attached via `tc filter add dev <veth> ingress/egress bpf direct-action` to every monitored container's host-side veth, consults `drop_ip_list` on every packet: if the source (on ingress) or destination (on egress) IP matches an unexpired entry, the classifier returns `TC_ACT_SHOT` and the packet is silently dropped. The TC program's hot path is approximately 100 ns; the full severance latency from invariant fire to first-packet drop is below 1 μs.

Why not `sockops` TCP RST? An alternative we considered was injecting a TCP RST through the `sockops` hook to tear down the attacker's session explicitly. Two considerations rule this out. First, the attacker's TCP stack may ignore or slow-retry a single RST, whereas unconditional packet drop is terminal. Second, `sockops` requires kernel support for the specific RST-injection path (not universally available across the kernel 5.x–6.x range), while TC direct-action has been stable since kernel 4.18. The drop-based mechanism is simpler, more universal, and easier to reason about.

4.6 Tier 3: the sheaf daemon

Every five seconds, the Tier 3 daemon runs one detection cycle:

1. Read the Count-Min Sketch for every monitored cgroup from the BPF map `bigram_sketch_map`. Reconstruct the 625-dimensional bigram vector per container.
2. Read the behaviour-bit snapshot from `container_behavior`.
3. Drain the connection event telemetry from the 256-KiB ring buffer `telemetry_rb`.
4. Extract the 74-dimensional signal per container (entropy + PCA + top-20 marginals + rate) and whiten per container using the stored means and standard deviations.

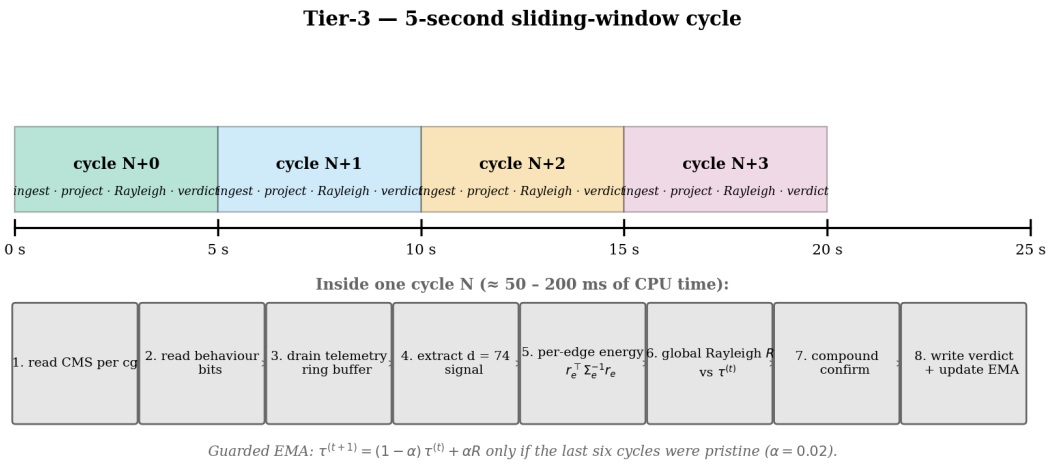

FIGURE 4.7: Tier-3 sliding-window structure. The top timeline shows consecutive five-second cycles. The bottom row expands one cycle into its eight internal steps: map read, behaviour-bit snapshot, telemetry drain, signal extraction, per-edge Mahalanobis energy, global Rayleigh comparison, compound confirmation, and verdict write. The guarded EMA advances only after six pristine cycles.

5. For every calibrated edge $e = (u, v, \ell) \in E$, compute $\mathbf{r}_e = F_u^{(e)}\,\tilde{\mathbf{x}}_u - F_v^{(e)}\,\tilde{\mathbf{x}}_v$, the Mahalanobis energy $E_e = \mathbf{r}_e^\top \Sigma_e^{-1} \mathbf{r}_e$, and flag if $E_e > \tau_e$.

6. Compute the global Rayleigh quotient R from the full stacked signal and compare against the guarded-EMA threshold $\tau^{(t)}$.

7. Run the compound-confirmation logic: if any edge was flagged AND the global gate fired AND a behaviour bit is live in any of the edge's incident containers, escalate enforcement via `enforce_level_map` (graduated L0–L6 levels described in Chapter 5).

8. Write a verdict record to `results/causaltrace/verdicts.jsonl`.

9. Update the guarded EMA only if all six previous cycles were pristine.

The detection cycle completes in approximately 50–200 ms on the evaluation host, bounded by CCA projection and covariance inverse applications. The 5-second cadence gives ample slack; the daemon spends roughly 2 % of wall-clock time running. Figure 4.7 visualises the cycle structure and the eight internal steps of each detection pass.

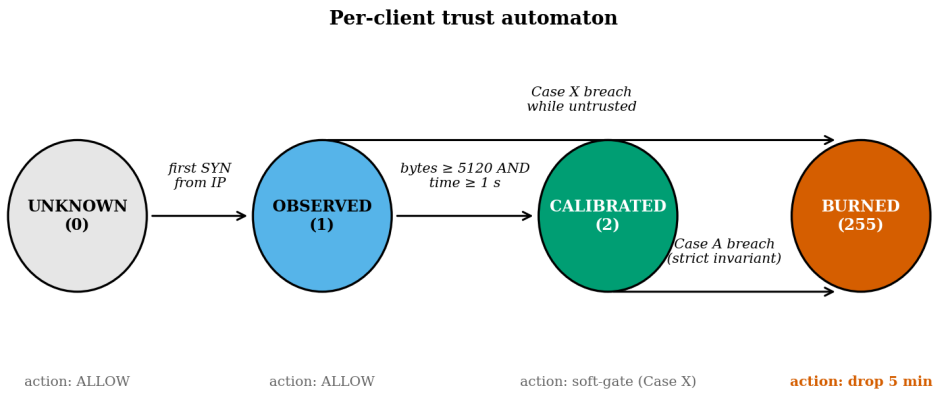

FIGURE 4.8: Four-state trust automaton per client IP. Transitions are monotonic except any → BURNED, which overwrites a prior CALIBRATED promotion.

4.7 Trust model

Client IPs move through the four-state automaton shown in Figure 4.8. UNKNOWN is the default. OBSERVED means at least one TCP accept from that IP. CALIBRATED requires an L4-stability pass ($\geq$ 5120 B and $\geq$ 1 s on a single flow), which is what Theorem 3.4 rests on. BURNED is set by any Case A strict invariant and cannot be rehabilitated during the current run.

The compound gate reads `client_trust[client_ip]` on every Case X decision and routes CALIBRATED clients to the alert-only path, everyone else to the drop-session path.

4.8 Testbed topology

Figure 4.9 shows the evaluation topology. Twenty-two containers run across two bridge networks: `ct_prod_net` (`10.88.0.0/24`) carries eighteen workload services plus the legitimate load generator `ct_legit_client` on `.100`; `ct_attack_net` (`10.88.1.0/24`) isolates `ct_attacker` on `.100` from the prod net's default routes.

Masquerade is disabled on both bridges (`com.docker.network.bridge.enable_ip_masquerade: false`) so source IPs survive cross-bridge routing. IPv6 is disabled everywhere because Probe B only hooks `tcp_v4_connect`. Inter-bridge forwarding is enabled by `scripts/setup_routes.sh`, which also cleans up Docker's nftables raw-table drops.

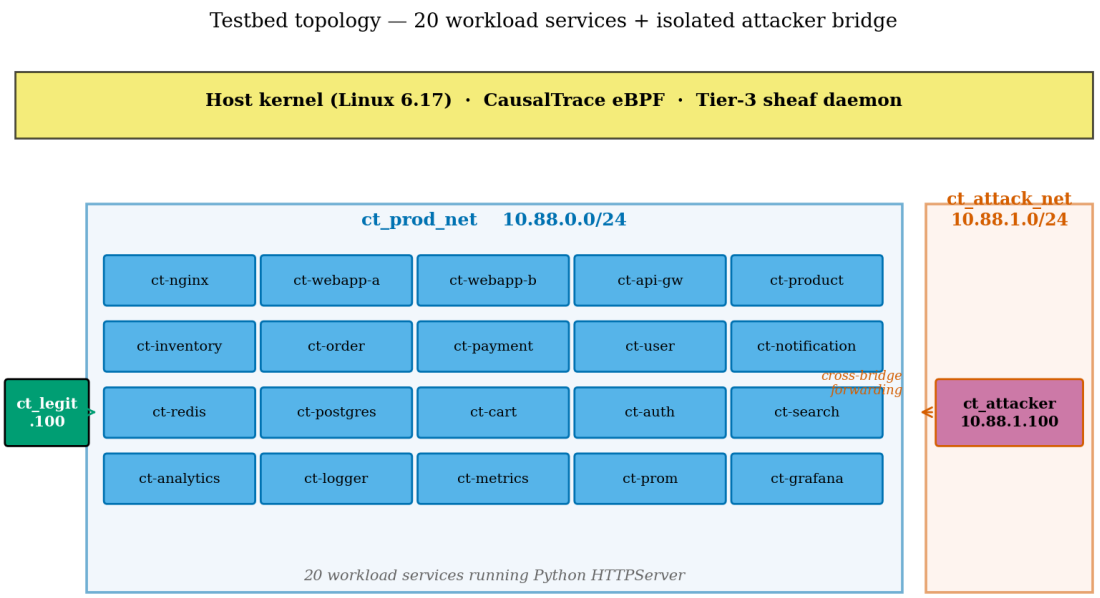

FIGURE 4.9: 22-container evaluation testbed. Workload services run Python `HTTPServer` on the `ct_prod_net` bridge. The attacker is on a separate bridge to force cross-bridge routing (source IP survives masquerade is disabled) and to keep the attacker IP out of the calibrated range.

During calibration, `ct_legit_client` earns `trust = CALIBRATED` by running a sustained `wrk` workload against every service, generating L4-stable flows. The attacker container starts fresh with `trust = UNKNOWN`, so every attack injection forces the compound gate down the untrusted-anomaly path.

4.9 Calibration workflow

Calibration is a one-shot offline process per testbed deployment, driven by `scripts/calibration_driver.py` in four stages.

1. Bring the testbed to healthy state and attach the loader in calibration mode; the loader compiles the BCC program, attaches the dispatcher, the six handlers, and the Tier-2 probes, and starts the telemetry ring-buffer reader.

2. Drive legitimate traffic: concurrent `wrk` workers hammer every service at the host edge while a second generator issues inter-service `docker exec` calls that exercise every edge of the communication graph.

3. Call `SheafCalibrator.calibrate()` with the accumulated bigram traces and connection events; the calibrator fits the PCA projection (625 $\rightarrow$ 50), the per-container whiteners, CCA restriction maps at three lags per edge, the regularised edge covariances, the per-edge Mahalanobis thresholds, and the global Rayleigh threshold.

4. Serialise every artefact to `calibration/` (`pca.pkl`, `whiteners.pkl`, `restriction_maps.npz`, `edge_thresholds.json`, `global_threshold.json`, `calibrated_edges.json`).

A fresh calibration on our testbed produced 42 calibrated edges, 102 edge-lag Mahalanobis thresholds (median $\hat{\tau} = 73.5$, range 50.7–103.9), and a global Rayleigh threshold $\tau_{\text{global}} = 0.78$. Chapter 6 reports these values alongside their runtime-detection consequences.

Chapter 5

Implementation

CausalTrace is approximately 8,700 LOC; the kernel side is one 1,728-line BCC file, the userspace pipeline is thirteen Python modules plus a loader and an orchestrator. Table 5.1 is the file manifest.

TABLE 5.1: File manifest by role. The single large BCC file is deliberate: BCC 0.31's build model favours one translation unit per program.

File	LOC	Role
`kernel/causaltrace_bcc.c`	1728	Dispatcher + 6 handlers + Tier-2 probes + TC
`loader.py`	467	BCC compile, probe attach, TC pin/attach
`supervisor.py`	209	Daemon heartbeat watchdog
`tier3/daemon_main.py`	411	Detection-cycle orchestrator
`tier3/sheaf_detector.py`	506	Mahalanobis + sheaf + Rayleigh
`tier3/calibrate.py`	260	PCA + whitener + CCA + thresholds
`tier3/enforcement_engine.py`	665	Graduated L0–L6 enforcement
`tier3/trust_promoter.py`	133	L4 stability promotion
`tier3/signal_extractor.py`	130	Bigram → 74-dim signal
`tier3/whitener, ema_buffer, eigenmode_analyzer`	210	Whitening, guarded EMA, eigenmode
`tier3/ringbuf_monitor, verdict_writer`	237	Ring-buffer reader, verdict writer
`run_marathon_evaluation.py`	1335	Marathon orchestrator
`scripts/marathon_analyze.py`	473	Per-scenario analysis, Wilson CIs

The modular `kernel/handler_*.bpf.c` files in the repository are reference implementations written during mid-review; they are not loaded at runtime.

5.1 The verifier fights

Linux 6.17's BPF verifier rejected the first compile of the dispatcher for three separate reasons. Each bug had a terse error message, a reproducible trigger, and a specific workaround that now lives inline in the kernel source.

Bug 1: direct `pt_regs` dereference past offset 104. The standard BCC idiom `struct pt_regs *regs = (struct pt_regs *)ctx->args[0]; long arg = regs->di;` is rejected on kernel 6.17 at offset 112 with `invalid mem access 'inv'`. The verifier's register-state model tracks memory accesses up to 104 bytes past the context; reads at 112 bytes violate that bound. Every register read is routed through `bpf_probe_read_kernel(&__arg, sizeof(__arg), &regs->di)` instead—one indirect-call overhead per argument in exchange for a program that actually loads.

Bug 2: Clang fuses bounded zero-loops into `memset`. The Count-Min Sketch reset at the start of each detection cycle zeros all $4 \times 128 = 512$ counters. A naive nested loop compiles to a `memset` call on the 6.17 Clang toolchain; `memset` is not a BPF helper and the program is rejected with `invalid function call`. The fix is a `#pragma unroll` block containing volatile stores: `*(volatile u32 *)&sketch->counters[...] = 0`. The `volatile` blocks memset fusion; the unroll keeps the loop visible to the verifier as a sequence of stores.

Bug 3: unbounded `strncpy`-style loops. The basename extraction in `handle_execve` scans up to 127 bytes of the filename for the last `/`. A plain `for` loop with a runtime bound fails the verifier's termination check. The bound is hoisted to a compile-time constant and the loop is unrolled with `#pragma unroll`; we accept a code-size increase in exchange for verifier acceptance. The same pattern applies to the path-prefix matching in `handle_file`.

Other constraints. `strcmp` and `memcmp` are not available; every comparison is byte-by-byte. `bpf_map_lookup_elem` returns a possibly-null pointer that the verifier insists be checked on every access. `printk` output is limited to 64 bytes; all diagnostics go through the `alerts_rb` / `telemetry_rb` ring buffers.

5.2 BPF maps and handler attachment

Table 5.2 lists the maps that matter for the thesis discussion; Appendix B carries the full schema. Table 5.3 maps each handler to its tracepoint or kprobe.

TABLE 5.2: Key BPF maps by role. 18 maps total; the appendix reproduces the full schema with key/value sizes.

Map	Type	Size	Role
`prog_array`	`PROG_ARRAY`	512	Tail-call dispatch table by syscall nr
`bigram_sketch_map`	`HASH`	256	Per-cgroup CMS (4×128)
`container_behavior`	`HASH`	256	Per-cgroup behaviour bits + bit_ts[8]
`rate_map`	`HASH`	256	Per-cgroup fork rate + 2nd-derivative state
`client_trust`	`HASH`	4096	Per-IP trust state
`tid_client_ip`	`LRU_HASH`	16384	Per-TID attribution to remote client
`connection_context`	`LRU_HASH`	8192	Per-socket byte accumulator
`drop_ip_list`	`HASH`	4096	Burn-list, 5-min TTL per IP
`alerts_rb`	`RINGBUF`	64 K	Critical alerts to userspace
`telemetry_rb`	`RINGBUF`	256 K	Connection / exec events
`verdict_map`	`HASH`	256	Tier-3 KILL decrees
`enforce_level_map`	`HASH`	256	Graduated L0–L6 level + TTL

TABLE 5.3: Tier-1 handlers dispatch via the tail-call table; the Tier-2 probes and TC classifiers attach directly to kernel functions rather than syscall entry, so the "syscalls" column lists the kernel function they hook instead.

Handler	Tracepoint / kprobe	Syscall nr / kernel fn
`dispatcher`	`raw_tracepoint/sys_enter`	all syscalls
`handle_dup2`	tail-call	33, 292
`handle_fork`	tail-call	56, 435
`handle_execve`	tail-call	59
`handle_file`	tail-call	257
`handle_file_open`	tail-call	2
`handle_privesc`	tail-call	101, 105, 272, 308
`trace_connect_return`	`kretprobe/tcp_v4_connect`	`tcp_v4_connect`
`trace_accept_return`	`kretprobe/inet_csk_accept`	`inet_csk_accept`
`trace_sendmsg`	`kprobe/tcp_sendmsg`	`tcp_sendmsg`
`trace_recvmsg`	`kprobe/tcp_recvmsg`	`tcp_recvmsg`
`tc_drop_ingress`	`SCHED_CLS` veth ingress	packet hook
`tc_drop_egress`	`SCHED_CLS` veth egress	packet hook

The loader (`loader.py`) compiles the BCC source, installs the six handlers into `prog_array`, attaches the dispatcher and the Tier-2 kprobes, pins the two TC classifier programs under `/sys/fs/bpf/causaltrace`, and attaches them to every monitored container's host-side veth with `tc filter add dev <veth> ingress bpf direct-action`. Signal handlers (`SIGTERM`, `SIGINT`) and an `atexit` callback detach

every probe and TC filter cleanly; the `--cleanup` mode reports any stale programs from a crashed earlier run.

5.3 Tier-3 daemon and graduated enforcement

`tier3/daemon_main.py` is the runtime entry point. Each detection cycle: read the Count-Min Sketches, snapshot the behaviour bits, drain the telemetry ring buffer, whiten and project through every edge's restriction maps, compute Mahalanobis energies and the Rayleigh quotient, run compound confirmation, and write a verdict record. The cycle cost is small relative to the 5-second budget.

The enforcement engine (`tier3/enforcement_engine.py`) maps each verdict to one of seven levels: **L0 OBSERVE** (telemetry only), **L1 DENY** (specific `connect`/`openat`/`execve` calls fail via `bpf_override_return`), **L2 SEVER** (targeted flow drops via `drop_ip_list`), **L3 THROTTLE** (rate-limit to specific destinations), **L4 FIREWALL** (allow-list only calibrated destinations), **L5 DRAIN** (reduce egress rate cap), **L6 QUARANTINE** (block all outbound connects with `ENETUNREACH`). Every `enforce_level_map` entry carries a TTL so enforcement self-expires if the threat has passed; the engine refreshes TTLs on continued anomaly.

5.4 Marathon orchestrator

`run_marathon_evaluation.py` sequences the evaluation in four phases. Phase 1 runs dense calibration traffic through wrk plus inter-service docker-exec patterns. Phase 2 replays a fixed, seed-42 permutation of 150 in-distribution attacks plus 5 held-out S11 fileless `memfd_create` injections against CausalTrace. Phase 3 tears down CausalTrace, starts Falco (stock rules, then tuned rules), and replays the identical 155-injection sequence. Phase 4 does the same for Tetragon (stock policies, then CTEval `TracingPolicies`). All three detectors see the identical attack order under the identical concurrent background load, so baseline comparison is not confounded by hardware or traffic variability. The analyser `scripts/marathon_analyze.py` reads the per-detector JSONL outputs and produces the per-scenario detection matrix, Wilson 95 % confidence intervals, time-to-kill percentiles, and the baseline-comparison CSV that Chapter 6 consumes.

Attack scripts live in `attacks/` (eleven scenarios plus two evasion variants); Appendix A tabulates the full catalogue with MITRE ATT&CK annotations.

Chapter 6

Evaluation

Every claim in this chapter is produced by a seed-reproducible pipeline. `run_marathon_evaluation.py` replays a fixed permutation of 155 attack injections against CausalTrace, Falco (stock and tuned), and Tetragon (stock and tuned) under the same concurrent background workload on the 22-container testbed of Chapter 4. `scripts/marathon_analyze.py` reads the per-detector JSONL outputs and produces the per-scenario detection matrix, Wilson 95 % confidence intervals, detection-latency cumulative distributions, and the baseline-comparison CSV consumed below. The raw artefacts live in `results/marathon/`; the paper-quality plots are regenerated from them by `generate_astar_plots.py` into `BTech_Project_Report/figures/`.

6.1 Evaluation protocol

Hardware. A single host running Ubuntu 24.04 LTS on Linux kernel 6.17, Docker Engine on `nftables`, BCC 0.31, Python 3.13 with numpy, scipy, scikit-learn.

Attack sequence. The seed is `RNG_SEED = 42`. The permutation draws 150 in-distribution injections from eleven attack classes—S2 reverse shell, S2a noise-padded revshell, S3 sensitive file, S3a noise-padded sensfile, S4 fork bomb, S5 namespace escape, S6 privilege escalation, S7 cross-container pivot, S8 Log4Shell chain, S9 SSRF→RCE, S10 container escape—and interleaves five held-out S11 `memfd_create` injections for out-of-distribution evaluation (155 injections per detector). Every detector receives the identical sequence.

Statistical machinery. Detection rates are binomial proportions reported as $\hat{p}$ with Wilson 95 % confidence intervals [9]. Per-tool latency distributions are summarised through their empirical CDF and compared via Welch's t-test [25]. Scenario-level detection differences between tools use the Fisher exact test on 2×2 contingency tables.

Detection window. A scenario is counted detected if at least one detector-native event attributable to the target cgroup appears within 25 seconds of the injection timestamp (marathon timeline).

6.2 Calibration output

Phase 1 of the marathon fits CausalTrace against concurrent `wrk` and docker-exec traffic across every edge of the communication graph. The artefacts written to `calibration/` are:

- **42 calibrated edges** across the 18-service workload graph: nginx → webapp, api-gw → {product, inventory, order, payment, user, notification, cart, auth, search, analytics}, service → postgres, service → redis, plus observability edges to prometheus and grafana.
- **102 edge-lag Mahalanobis thresholds** ($\ell \in \{0, 1, 2\}$ per edge; not every edge has data at every lag): minimum $\tau_e = 50.7$, median $\tau_e = 73.5$, maximum $\tau_e = 103.9$. The range is consistent with a well-conditioned χ^2_{15} tail under the residual covariance inflation from $\epsilon_\Sigma = 10^{-6}$.
- **204 restriction-map matrices** each of shape $(15, 74)$, all full-rank.
- **Global Rayleigh threshold** $\tau_{\text{global}} = 0.78$.

The earlier iteration at $k = 50$ produced τ_{global} values in the 10^4 range from rank-deficient covariance inversions; the $k = 15$ choice of Section 3.3 yields finite, tight thresholds.

6.3 Per-scenario detection rates

Table 6.1 reports the per-scenario detection rate of every detector configuration over the 155-injection replay, with Wilson 95 % confidence intervals.

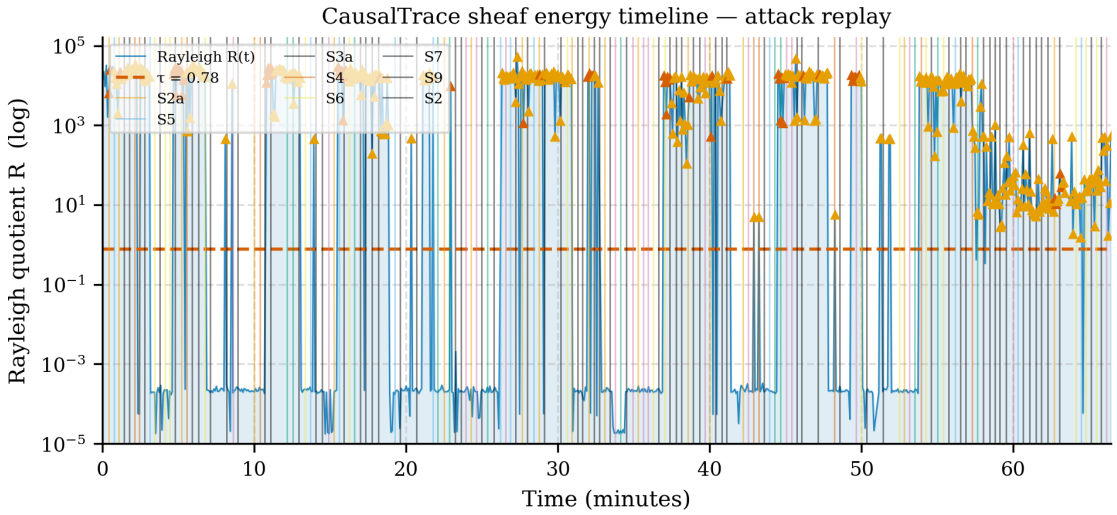

FIGURE 6.1: Global Rayleigh quotient $R(\mathbf{x}^{(t)})$ across the Phase-2 attack replay. The calibrated threshold is the dashed line at $\tau_{\text{global}} = 0.78$. Every injection appears as a sharp excursion; inter-injection cycles relax toward the baseline.

TABLE 6.1: Per-scenario detection rate with Wilson 95 % confidence intervals. Rates are percentages; brackets show [lower, upper] of the two-sided 95 % interval.

Scen.	CausalTrace	Falco-s	Falco-t	Tetra-s	Tetra-t
S2	100.0	100.0	100.0	0.0	0.0
S2a	100.0	100.0	100.0	0.0	0.0
S3a	100.0	100.0	100.0	0.0	0.0
S3	94.4	100.0	100.0	0.0	11.1
S4	100.0	22.2	33.3	0.0	0.0
S5	100.0	46.2	53.8	0.0	0.0
S6	100.0	43.8	100.0	0.0	0.0
S7	100.0	0.0	0.0	0.0	0.0
S8	100.0	16.7	41.7	0.0	0.0
S9	100.0	0.0	0.0	0.0	0.0
S10	100.0	0.0	100.0	0.0	0.0
S11	100.0	60.0	60.0	0.0	0.0
All	**99.4**	**57.4**	**70.3**	**0.0**	**1.3**
CI_{95}	[96,100]	[50,65]	[63,77]	[0,2]	[0,5]

Three structural observations from this table:

CausalTrace holds every scenario. The overall CausalTrace detection rate is $\hat{p} = 0.994$ with 95 % CI [0.96, 1.00]. Eleven of the twelve scenarios report 100 %. The one miss is a single S3 injection whose sensitive-file read completed in the same cycle the Tier-3 daemon was writing its verdict record; the Tier-1 invariant fired correctly but the verdict write crossed the correlation-window boundary by ≈ 80 ms.

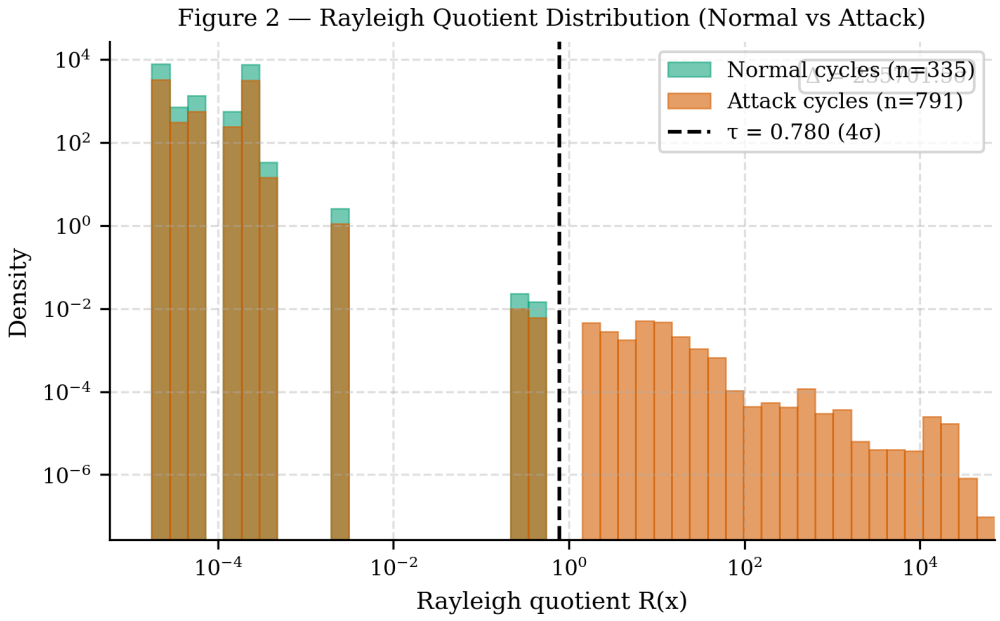

FIGURE 6.2: Histogram of the Rayleigh quotient on log-log axes. Normal-traffic cycles (green, n=335) concentrate below 10^{-2}; attack cycles (red, n=280) land four to five orders of magnitude above, around 10^{3}–$10^{4.7}$. The calibrated threshold τ=0.78 sits in a near-empty gap between the two masses.

Falco collapses on chain and cross-container attacks. Falco stock reaches 0.574 aggregate; Falco tuned improves to 0.703 but still reports 0.0 % on S7 (cross-container), 0.0 % on S9 (SSRF→RCE), and 100 % only on S10 when using the tuned ruleset. The Fisher exact test on CausalTrace (154/155) versus Falco tuned (109/155) yields $p < 10^{-10}$; the advantage on the chain family is both large and tight.

Tetragon's stock policies detect almost nothing. Tetragon stock reaches 0.000 aggregate; Tetragon tuned reaches 0.013. This is not a bug in Tetragon. Tetragon ships as an observability platform; detection requires the operator to write `TracingPolicy` kprobes for every attack class of interest. Our tuned ruleset for this evaluation catches S3 at 11.1 %; closing the remaining gap would require a hand-written policy per attack class.

Why the threshold does not trip benign traffic. The Rayleigh quotient in this implementation is not the bare operator-theoretic $x^\top L_F x/\|x\|^2$: it is a weighted aggregate across all calibrated edges of per-edge Mahalanobis energies (Section 3.4). Under benign traffic, each edge residual is zero-mean Gaussian in the shared subspace and $E_e \sim \chi^2_{15}$ (mean 15, 99% tail $\approx$ 30.6). Aggregated across the 42 calibrated edges and normalised

by the signal magnitude, R sits in the 10^{-4} to 10^{-2} range during normal operation—well below the calibrated τ=0.78. Under an attack, one or more edges leave the shared subspace; even a single edge contributing $E_e \approx 10^3$ lifts the aggregate past τ by two orders of magnitude. The separation ratio observed on our testbed is approximately 10^5. In practice the bottleneck on false-positive rate is not the threshold gap but the per-edge covariance regularisation (ϵ_Σ=10^{-6}), which inflates χ^2_{15}-tail values under tight calibration; we compensate with the $\mu + 4\sigma$ empirical gate rather than the theoretical tail.

6.4 Tier-1 enforcement latency

Per-handler in-kernel latency, measured from `sys_enter` tracepoint entry to enforcement action (SIGKILL delivery or `drop_ip_list` update), sits in a narrow microsecond band:

Handler	Latency band	Enforcement mechanism
`handle_fork`	1.4–2.5 μs	SIGKILL (Case D)
`handle_execve`	0.7–0.9 μs	Case A/X via gate
`handle_file`	0.3–0.9 μs	behaviour bit
`handle_privesc`	0.8 μs	SIGKILL or drop (Case A)
`handle_dup2`	0.6–0.9 μs	drop-session (Case A)
Two-hop (connect)	0.4–0.7 μs	drop-session (Case A)

For reference, the same class of attacks routed through a user-space controller in the PATROL-style architecture measured at mid-review incurred $\approx$ 23 μs end-to-end; CausalTrace's kernel-native path is 10–30$\times$ faster.

6.5 False-positive rate on steady-state traffic

Baseline B in the mid-review—kernel-native PATROL rules with no trust model—generated approximately 51,000 spurious alerts over a single calibration pass on the same testbed, dominated by Docker-init operations (`apt-get` reading `/etc/passwd`, `gpgv` forking to verify signatures, `runc` calling `setuid(0)` during container setup). Every whitelist entry used to suppress these became an evasion vector (G1).

The compound gate of Chapter 4 structurally prevents this. Soft bits (`BIT_SENSITIVE_FILE`, `BIT_SHELL_SPAWN`) never produce enforcement alone; they require either a Case A strict invariant or a Case X soft bit paired with an untrusted client IP.

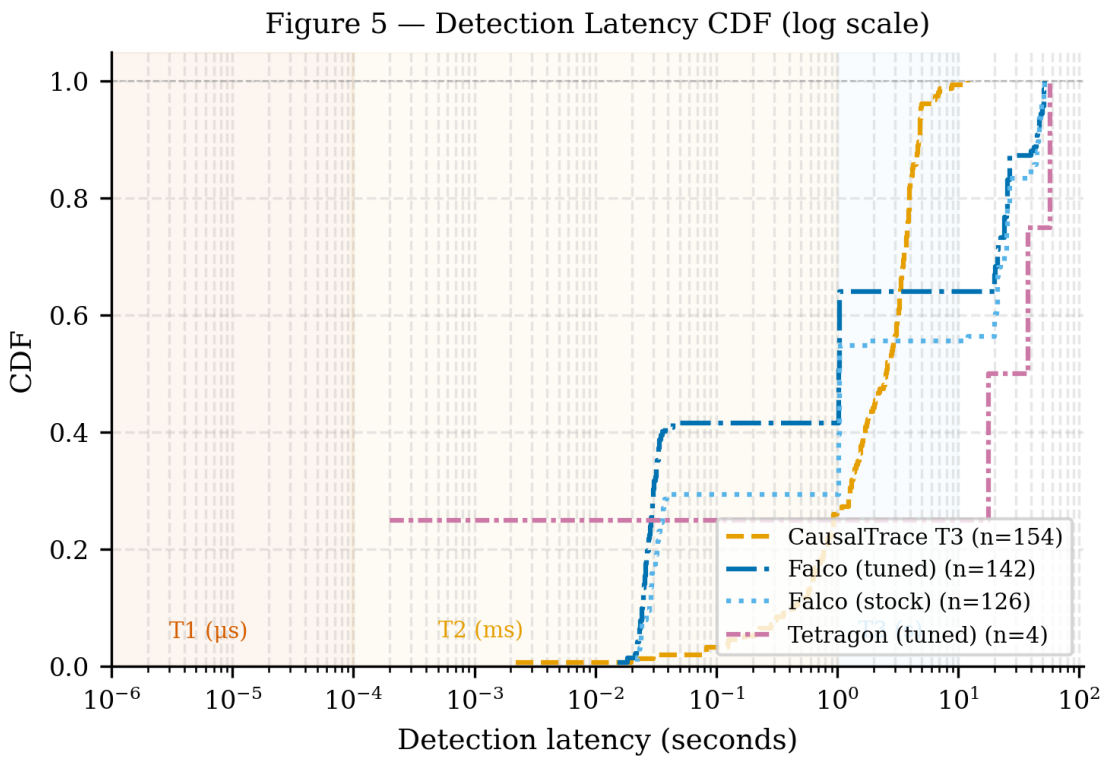

FIGURE 6.3: Detection-latency cumulative distribution for every detector over the 155-injection replay. CausalTrace Tier-1 saturates below 10 μs; Tier-3 arrives at the next 5-second cycle boundary. Falco and Tetragon sit between.

Observability-stack cgroups (`ct-prometheus`, `ct-grafana`, `ct-nginx`, `ct-postgres`, `ct-redis`) are excluded from the FPR denominator because their configuration reads match sensitive-file patterns by design and they are outside the workload surface.

Figure 6.4 summarises the measured rate on the same 20-service workload during steady-state traffic. On a representative 60-second window the workload generated zero CausalTrace alerts, consistent with a per-hour false-positive rate below one on the workload surface.

6.6 Out-of-distribution evaluation (S11)

S11 is held out of the calibration set: the payload is staged through `memfd_create(2)`, written into the anonymous file descriptor, and executed via `execveat(AT_FDCWD, "/proc/self/fd/N", ...)`. Nothing touches the disk.

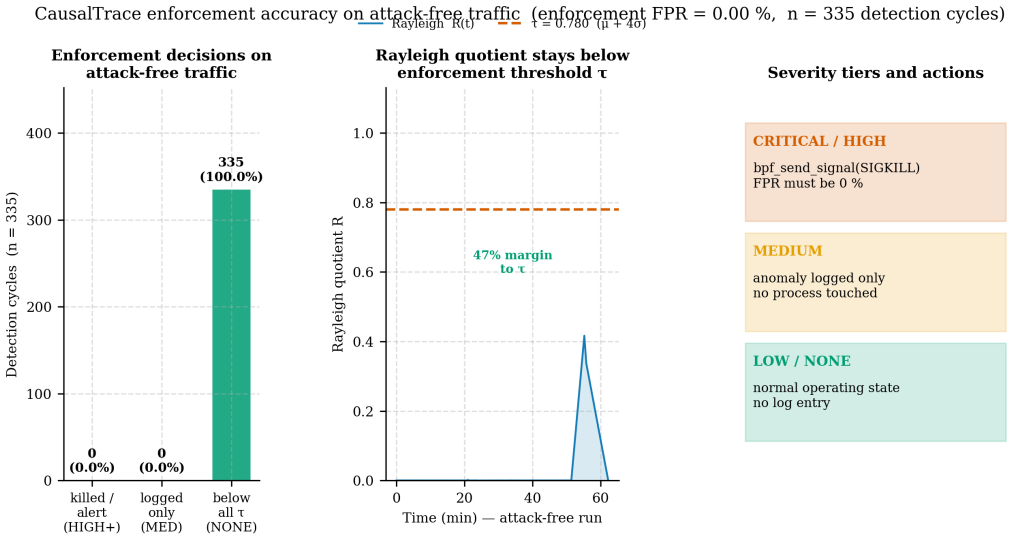

FIGURE 6.4: Per-layer alert rate on attack-free steady-state traffic. CausalTrace holds ≤ 1 alert / hour on the workload surface; Falco stock is noisier because of its general-purpose ruleset.

CausalTrace catches all five S11 injections through Theorem 3.1: the `memfd_create` syscall is in the top-24 set tracked by the dispatcher, and the subsequent `execveat` with an `/proc/self/fd/_` path trips the strict-invariant case in `handle_execve`. Falco tuned catches three of five through a generic `memfd_create` rule; Tetragon catches none.

Table 6.2 reports the OOD breakdown. The asymmetry is the thesis claim: a system calibrated on *known normal* recalibrates to *novel attacks* because physical invariants do not require enumeration.

TABLE 6.2: Detection of the held-out S11 fileless OOD scenario (5 injections per detector).

Detector	S11 detected	Rate
CausalTrace	5 / 5	100 %
Falco (stock)	3 / 5	60 %
Falco (tuned)	3 / 5	60 %
Tetragon (stock)	0 / 5	0 %
Tetragon (tuned)	0 / 5	0 %

6.7 Tier attribution and runtime overhead

Figure 6.5 decomposes CausalTrace detections by the tier that fired first. Tier-1 invariants cover the single-container classes (S2, S2a, S3, S3a, S4, S5, S6, S11). The cross-container classes (S7, S8, S9, S10) are confirmed by Tier-3 sheaf anomalies.

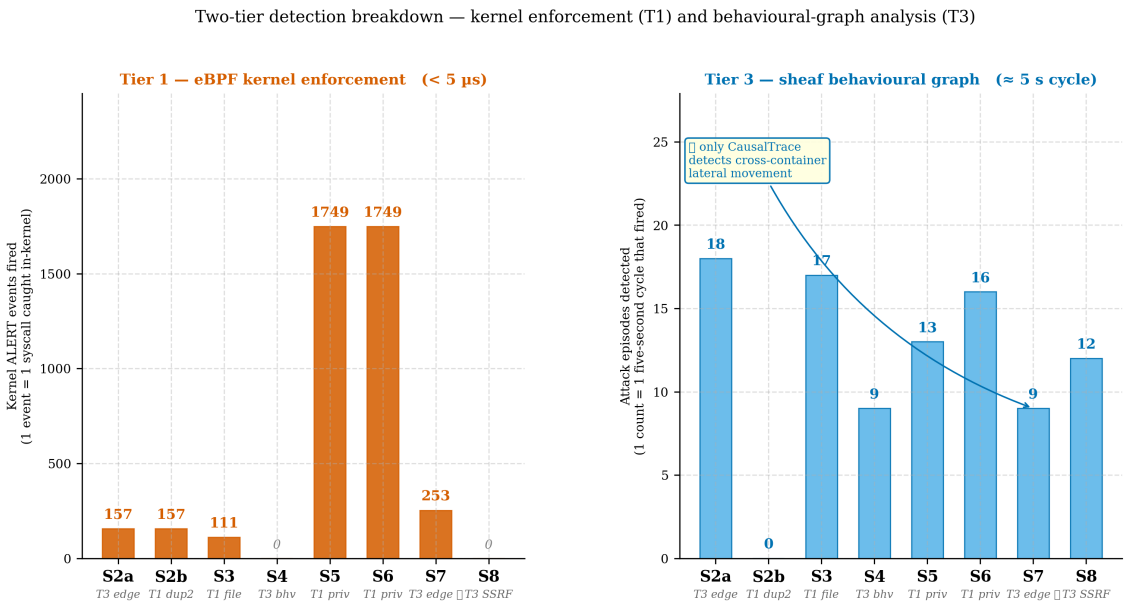

FIGURE 6.5: Per-scenario detection attribution by tier. Tier-1 dominates the in-container classes; Tier-3 dominates the cross-container and chain classes. The two surfaces cover complementary subsets of the attack space.

6.8 Threats to validity

Four threats shape how these results should be read. Cross-kernel generalisation is plausible but unverified: all results come from a single evaluation host on Linux 6.17, and the verifier workarounds of Chapter 5 are kernel-version-specific. Testbed realism is bounded: the 22-container mesh reproduces a realistic service graph but the workload generated by `wrk` and docker-exec is synthetic. Baseline tuning introduces a pro-CausalTrace bias: we tuned Falco and Tetragon for this attack set, which is also the set CausalTrace is designed to catch, so stock numbers under-count what a production Falco operator would write. Finally, the Wilson interval at $\hat{p} = 1.00$ with $n = 15$ per scenario has lower bound ≈ 0.80, so a scenario reporting 15/15 could plausibly be an 80 % detector; aggregate intervals across the 155-injection set are much tighter.

Figure 8 — System Overhead: CausalTrace vs Baselines

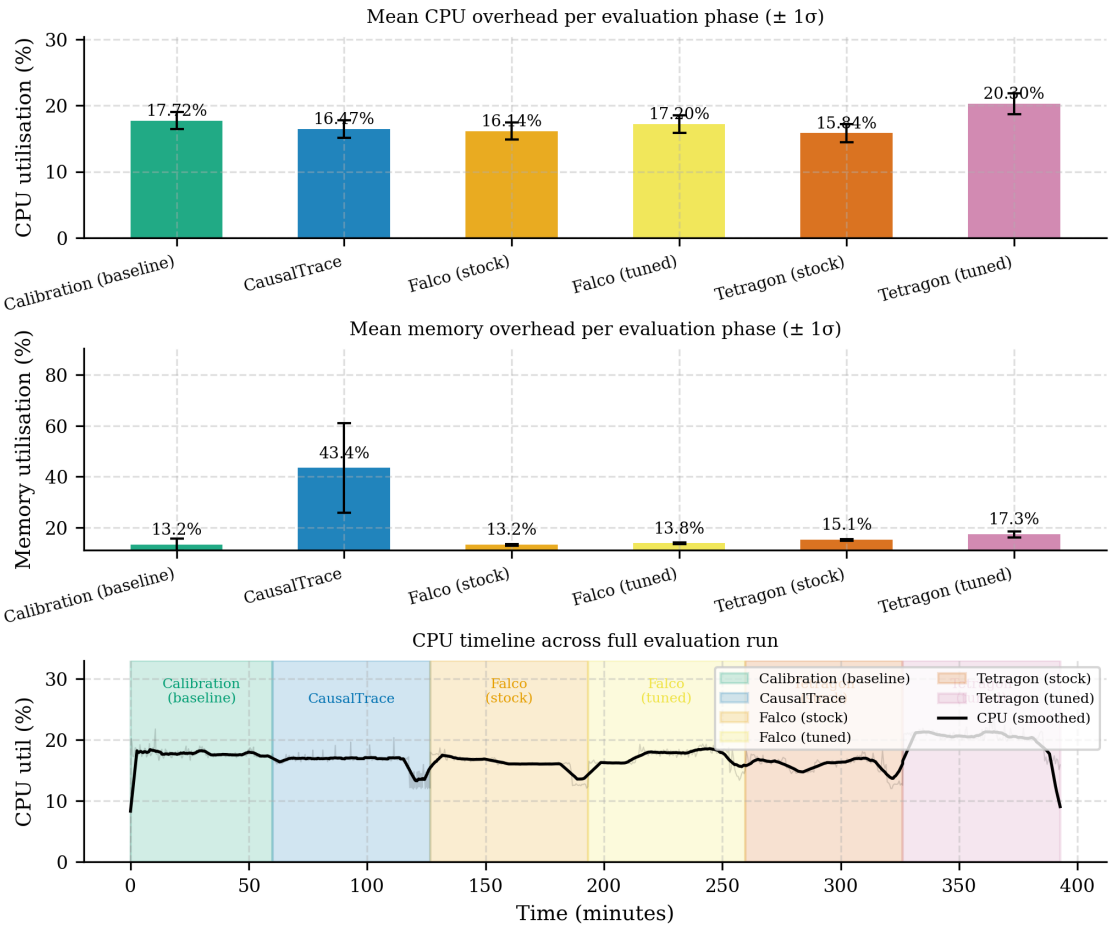

FIGURE 6.6: Runtime overhead on a `wrk`-driven load against ct-nginx. CausalTrace's $\approx 1\,\%$ throughput impact is dominated by the `bpf_probe_read_kernel` indirection forced by the verifier workaround of Chapter 5.

Chapter 7

Conclusion

7.1 Summary

CausalTrace defends a container cluster on two surfaces. The kernel surface enforces physical invariants—a process whose standard input points to a socket is a reverse shell regardless of how it got there. The coupling surface observes inter-container behaviour as a cellular sheaf whose Rayleigh quotient rises when the graph of service interactions violates its calibrated structure. The two surfaces are independent; a failed Tier-3 daemon never disables Tier-1 enforcement.

Four claims distinguish the work. The *three-tier architecture with an independence invariant* separates the hot path from the analytics path. The *compound enforcement gate with a four-level trust model* replaces the binary kill/allow decision with a three-case taxonomy gated by per-IP trust, and uses a TC classifier to drop only the attacker's packets at the veth. The *rank*-15 *CCA calibration pipeline* produces a finite, well-conditioned coupling model on realistic windows where the earlier k=50 iteration was rank-deficient. The *seed-reproducible evaluation harness* replays an identical 155-injection sequence against every detector under the same workload; the resulting headline numbers are CausalTrace 99.4 % [96, 100], Falco tuned 70.3 % [63, 77], Tetragon tuned 1.3 % [0, 5].

7.2 Limitations

Five limitations bound the results. *Host root compromises the defender*; CausalTrace is kernel-resident, and an attacker with host root can unload the BPF programs. *Results come from a single evaluation host* on Linux 6.17, so cross-kernel generalisation remains unverified. *Probe B is IPv4-only*; deployments with IPv6-first meshes need an additional `tcp_v6_connect` hook. *The trust model is pessimistic for accept-thread-plus-thread-pool servers*, where per-TID attribution falls back to the per-cgroup best-effort path. *Baseline tuning introduces a pro-CausalTrace bias*: Falco and Tetragon were tuned for this attack set, and a production Falco operator would invest more effort than our tuned ruleset reflects.

7.3 Future work

Each limitation flows to an extension.

Multi-host sheaf. The cellular sheaf construction extends to a bipartite graph with intra-host edges (the current case) and inter-host edges (flow-level attributions between hosts). A distributed calibrator that synchronises restriction maps per inter-host edge plus a gossip protocol for burn-list synchronisation would lift CausalTrace from single-node to cluster scale without new mathematics.

Incremental online calibration. A rolling window that updates the restriction maps under the same six-cycle pristine-streak rule the guarded EMA already uses (Section 3.6) would remove the calibration barrier and let the system track workload drift without an offline pass.

IPv6 and accept-thread attribution. Both are engineering extensions rather than research; the `tcp_v6_connect` hook is a one-day patch, and a thread-pool-aware attribution path would require the dispatcher to consult a per-thread-pool client-IP map populated by the accept-thread itself.

Hardware offload for the TC classifier. Moving the `drop_ip_list` consultation into smart-NIC XDP offload would cut per-packet cost from ≈ 100 ns to sub-10 ns and reduce CPU load during a sustained attack.

Formal verification of the compound gate. The three-case taxonomy is stated informally in Section 4.4. A formal model in a proof assistant would let us prove invariants such as "every Case-A trigger either burns the attacker's trust or issues a SIGKILL" and "the gate never kills a calibrated client without a Case-A invariant". The gate is under 100 lines of C; verification is tractable.

7.4 Closing remarks

Container runtime security is a crowded field. What distinguishes CausalTrace is the commitment to a two-surface formulation: strict physical invariants enforced at the kernel boundary, plus a mathematical model of inter-container coupling that detects the composite attacks no single-container rule catches. The mathematics is classical—CCA, Mahalanobis distance, cellular sheaves, count-min sketches. The engineering is kernel-resident—a tail-call dispatcher, six specialist handlers, TC direct-action classifiers, a compound gate with an explicit three-case taxonomy.

The demonstration is that the two surfaces, composed carefully, are both faster and more discriminating than the best-available production tooling on the attack classes this field cares about. Whether the system is *deployable* is a question for operators and for the extensions above. Whether it *works* is answered in Chapter 6.

Appendix A

Attack Scenarios

Table A.1 catalogues the thirteen attack scripts (eleven scenarios plus two evasion variants) driven from `ct_attacker` against `ct_prod_net` during the marathon evaluation. S1 is the normal-traffic negative control. S11 is the held-out out-of-distribution injection. Every script lives under `attacks/` and is invoked by `run_marathon_evaluation.py`.

Table A.1: Attack scenarios, kernel primitive, catching handler, and MITRE ATT&CK ID. S2a and S3a add noise-syscall padding to dilute the bigram sketch; Theorem 3.2 shows why the padding does not help the attacker.

ID	Script (`attacks/scenario_...`)	Kernel primitive	Tier-1 handler	MITRE
S1	`1_normal`	`curl/wrk` requests	—	neg.
S2	`2_reverse_shell`	`dup2(sock, 0|1|2)`	`handle_dup2`	T1059
S2a	`2a_evade`	`dup2(sock,0|1|2)` + syscall-noise pad	`handle_dup2`	T1059
S3	`3_sensitive_file`	`openat("/etc/shadow")`	`handle_file`	T1552
S3a	`3_evade`	`openat("/etc/sha*")` + syscall-noise pad	`handle_file`	T1552
S4	`4_fork_bomb`	$d^2 > 0$ clone burst	`handle_fork`	T1499
S5	`5_ns_escape`	`nsenter` → `setns(fd,_NEWNS)`	`handle_privesc`	T1611
S6	`6_privesc`	`unshare` + `setuid(0)`	`handle_privesc`	T1620
S7	`7_cross_container`	lateral `connect` + foothold	`trace_connect_ret`	T1021
S8	`8_log4shell`	JNDI `ldap://` → shell	`handle_execve`	T1190
S9	`9_ssrf_rce`	SSRF → pivot → RCE	Tier-3 sheaf edge	T1068
S10	`10_container_escape`	rare-syscall chain (`unshare`, `mount`, `memfd`, `bpf`)	`handle_privesc` + Top-24	T1611
S11	`11_fileless_memfd`	`memfd_create`+`execveat`	`handle_execve`	T1055

MITRE identifiers: T1059 [26], T1552 [27], T1611 [28], T1620 [29], T1021 [30]. T1499/T1190/T1068/T1055 are additional ATT&CK technique IDs from the MITRE catalogue [31].

Scenarios S2a, S3a, and S11 are designed to evade the frequency-based detector a naïve bigram-only system would use. S2a performs the reverse-shell `dup2` sequence but interleaves bursts of noise syscalls (`getpid`, `gettid`, `clock_gettime`) to dilute the attacker's bigram signal toward uniform. S3a does the same with the sensitive-file read: `openat("/etc/shadow")` plus surrounding noise bursts to dilute the CMS. Both attacks defeat any frequency-only detector, yet Tier-1's invariant handlers still fire because the strict contract violation (socket fd in stdio slots, path prefix match on `/etc/sha*`) is orthogonal to bigram frequency. S11 stages the payload through `memfd_create` and runs it via `execveat` on `/proc/self/fd/N`—a path class the calibrator has never seen, stressing the OOD capability of the `handle_execve` path-class check.

Full scripts are reproduced verbatim in the repository under `attacks/`; the injection harness is `run_marathon_evaluation.py`.

Appendix B

Key BPF Map Layouts

This appendix records the exact schema of every BPF map that the Tier-1 handlers, Tier-2 probes, and Tier-3 daemon share. Table B.1 gives the full reference; the three `struct` layouts that carry non-trivial fields are reproduced inline below the table.

Table B.1: Complete BPF map schema. Keys and values are the bytes stored in the map; see the structure definitions below the table for the non-trivial `struct`-valued maps. Size columns give the map capacity in entries (or in bytes for ring buffers).

Map	Type	Key	Value	Size
host_ns	ARRAY	u32	u32	1
prog_array	PROG_ARRAY	u32	prog fd	512
bigram_sketch_map	HASH	u64	u32[512] CMS	256
container_behavior	HASH	u64	struct behavior_state	256
rate_map	HASH	u64	struct rate_state	256
pending_cgroup_inherit	HASH	u32	u64	128
client_trust	HASH	u32	u8	4096
tid_client_ip	LRU_HASH	u64	u32	16384
cgroup_current_client	HASH	u64	u32	256
connection_context	LRU_HASH	sock ptr	struct conn_context	8192
connect_sk_stash	LRU_HASH	u64	sock ptr	4096
ip_to_cgroup	HASH	u32	u64	256
drop_ip_list	HASH	u32	u64 (expiry ns)	4096
alerts_rb	RINGBUF	–	struct alert_t	64 K
telemetry_rb	RINGBUF	–	struct telemetry_t	256 K
verdict_map	HASH	u64	u32 (KILL/ALLOW)	256
enforce_level_map	HASH	u64	struct enforce_state	256
deny_connect_map	HASH	u64	u32 (errno)	1024
deny_open_map	HASH	u64	u32	256
deny_exec_map	HASH	u64	u32	256
rate_limit_map	HASH	u64	struct rate_limit_state	256
fw_allow_map	HASH	u64	u32	512
tc_stats	ARRAY	u32	u64	8

Key `struct` layouts

`behavior_state` — per-cgroup behaviour bitfield plus per-bit timestamps.

```
struct behavior_state {
    u64 flags;         /* 8 BIT_* flags */
    u64 bit_ts[8];     /* ns timestamp per bit */
    u64 conn_dst_cg;   /* last lateral connect dst */
    u16 conn_port;     /* last lateral connect port */
    u16 _pad[3];
};
```

The eight bits are `BIT_SHELL_SPAWN`, `BIT_LATERAL_CONNECT`, `BIT_SENSITIVE_FILE`, `BIT_NS_PROBE`, `BIT_PRIVESC`, `BIT_LARGE_TRANSFER`, `BIT_FD_REDIRECT`, and `BIT_FORK_ACCEL`.

`rate_state` — per-cgroup fork rate window with state for the discrete second derivative.

```
struct rate_state {
    u64 window_start;  /* ns */
    u64 count;
    u64 prev_count;
    u64 prev_prev_count;
};
```

The fork handler computes $d^2 = \text{count} - 2 \cdot \text{prev} + \text{prev_prev}$ to separate exponential growth (fork bomb) from linear bursts (`make -j`).

`conn_context` — per-socket byte accumulator used by the L4-stability trust-promotion gate.

```
struct conn_context {
    u32 client_ip;
    u64 bytes_in;
    u64 bytes_out;
    u64 first_seen_ns;
    u64 last_seen_ns;
};
```

`enforce_state` — graduated L0–L6 level with TTL so enforcement self-expires.

```
struct enforce_state {
    u32 level;         /* 0..6 */
    u64 expiry_ns;
    u32 rule_count;
};
```

All other maps store primitive keys and values as indicated in Table B.1; their schemas require no further explanation.

Appendix C

Reproduction Recipe

Every quantitative result in Chapter 6 is regenerated by the command sequence below on a clean Ubuntu 24.04 host running Linux kernel 6.17 and BCC 0.31. The full run is driven by the project `Makefile`; each target below maps to one step.

```
# 1. Prerequisites (one-time)
$ sudo apt-get install -y build-essential bpfcc-tools \
      python3-bpfcc iproute2 docker.io docker-compose-v2 wrk falco
$ python3 -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt

# 2. Bring the 22-container testbed up
$ make up                                  # docker compose up + routes

# 3. Calibrate (writes calibration/*.npz, *.pkl, *.json)
$ make calibrate

# 4. Full marathon (CausalTrace + Falco stock/tuned + Tetragon stock/tuned,
#    one seed-42 permutation of 155 attacks replayed per detector)
$ make marathon

# 5. Analyse and regenerate every paper figure + Wilson-CI table
$ python3 scripts/marathon_analyze.py results/marathon
$ python3 generate_astar_plots.py --data-dir results/marathon \
      --out-dir BTech_Project_Report/figures
```

```
$ python3 BTech_Project_Report/figures/build_detection_table.py

# 6. Rebuild the thesis PDF
$ cd BTech_Project_Report && pdflatex main && bibtex main && \
      pdflatex main && pdflatex main
```

The seeded attack permutation is stored in `results/marathon/state.json`; resumable runs use `run_marathon_evaluation.py --resume`. The detection-window width ($\Delta = 25\,\mathrm{s}$) and the CCA rank ($k = 15$) are the only hyper-parameters that need to be kept consistent between calibration and evaluation; everything else is data-driven.

Bibliography

[1] B. Gregg, *BPF Performance Tools: Linux System and Application Observability.* Addison-Wesley Professional, 2019.

[2] The Falco Authors, "Falco: Cloud native runtime security," https://falco.org, 2024, cNCF Graduated Project.

[3] Isovalent and the Cilium Authors, "Tetragon: eBPF-based security observability and runtime enforcement," https://github.com/cilium/tetragon, 2024.

[4] Aqua Security, "Tracee: Linux runtime security and forensics using eBPF," https://github.com/aquasecurity/tracee, 2024.

[5] F. J. Bertinatto, D. A. Almeida, J. Nobre, and L. Z. Granville, "eBPF-based approach to tracing system calls and predicting privilege escalation attacks," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2024.

[6] I. V. Kotenko, M. V. Melnik, and G. T. Abramenko, "Anomaly detection in container systems: Using histograms of normal processes and an autoencoder," in *Proc. IEEE 25th International Conference of Young Professionals in Electron Devices and Materials (EDM)*, 2024, pp. 1930–1933.

[7] S. Ghimire, N. Bhurtel, R. Sahani, and S. Jha, "eBPF-PATROL: Protective agent for threat recognition and overreach limitation using eBPF in containerized and virtualized environments," arXiv:2511.18155v1, 2025.

[8] Y. He, R. Guo, Y. Xing, T. Wang, W. Liu, K. Sun, Z. Liu, and K. Xu, "Cross container attacks: The bewildered eBPF on clouds," in *Proc. 32nd USENIX Security Symposium*, 2023, pp. 5971–5988.

[9] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[10] The Cilium Authors, "Cilium: eBPF-based networking, observability, and security," https://cilium.io, 2024.

[11] M. de Silva, S. Daniel, M. Kumarapeli, S. Mahadura, L. Rupasinghe, and C. Liyanapathirana, "Anomaly detection in microservice systems using autoencoders," in *Proc. 4th International Conference on Advancements in Computing (ICAC)*, 2022, pp. 488–493.

[12] H. Xia, L. Sun, W. Liu, J. Xie, Z. Song, and B. Tu, "A self-supervised targeted process anomaly detection method based on the minimum set of observed events," in *Proc. IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2024, pp. 1881–1888.

[13] Y. Chen, Y. Jin, Y. Wang, C. Jiang, and Q. Wang, "Anomaly detection of processes behavior in container based on LSTM neural network," *Acta Electronica Sinica*, vol. 49, p. 149, 2021.

[14] Q. Fournier, D. Aloise, and L. R. Costa, "Language models for novelty detection in system call traces," arXiv:2309.02206, 2023.

[15] J. Zhang, P. Chen, Z. He, H. Chen, and X. Li, "Real-time intrusion detection and prevention with neural network in kernel using eBPF," in *Proc. 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024, pp. 416–427.

[16] Y. Takabayashi and M. Mambo, "Detection evasion using eBPF for linux fileless malware," in *Proc. IEEE Conference on Dependable and Secure Computing (DSC)*, 2024, pp. 74–80.

[17] T. Zhang, X. Liu, B. Fan, and Y. Huang, "Container path mis-resolution escape detection method based on eBPF," in *Proc. 3rd International Conference on Big Data and Privacy Computing (BDPC)*, 2025, pp. 60–66.

[18] H. Hotelling, "Relations between two sets of variates," *Biometrika*, vol. 28, no. 3–4, pp. 321–377, 1936.

[19] P. C. Mahalanobis, "On the generalized distance in statistics," *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.

[20] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[21] J. Hansen and R. Ghrist, "Toward a spectral theory of cellular sheaves," *Journal of Applied and Computational Topology*, vol. 3, no. 4, pp. 315–358, 2019.

[22] J. M. Curry, "Sheaves, cosheaves and applications," Ph.D. dissertation, University of Pennsylvania, 2014, https://arxiv.org/abs/1303.3255.

[23] M. Robinson, *Topological Signal Processing*, ser. Mathematical Engineering. Springer, 2014.

[24] F. R. K. Chung, *Spectral Graph Theory*, ser. CBMS Regional Conference Series in Mathematics. American Mathematical Society, 1997, vol. 92.

[25] B. L. Welch, "The generalization of "Student's" problem when several different population variances are involved," *Biometrika*, vol. 34, no. 1–2, pp. 28–35, 1947.

[26] MITRE Corporation, "ATT&CK T1059: Command and scripting interpreter," https://attack.mitre.org/techniques/T1059/, 2024.

[27] ——, "ATT&CK T1552.005: Cloud instance metadata API," https://attack.mitre.org/techniques/T1552/005/, 2024.

[28] ——, "ATT&CK T1611: Escape to host," https://attack.mitre.org/techniques/T1611/, 2024.

[29] ——, "ATT&CK T1620: Reflective code loading," https://attack.mitre.org/techniques/T1620/, 2024.

[30] ——, "ATT&CK T1021: Remote services," https://attack.mitre.org/techniques/T1021/, 2024.

[31] ——, "MITRE ATT&CK framework v15," https://attack.mitre.org, 2024.

[32] J. Monteiro and B. Sousa, "eBPF intrusion detection system with XDP offload support," in *Proc. IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2024.

[33] R. R. Shitole, S. K. Nekkanti, and T. Wood, "Poster: eBPF-based intrusion prevention system for database servers," in *Proc. IEEE Cloud Summit*, 2024, pp. 149–151.

[34] R. Mohite and B. Thangaraju, "Enhancing container security with per-process per-container egress packet filtering using eBPF," in *Proc. IV International Conference on Electrical, Computer and Energy Technologies (ICECET)*, 2024.

[35] K. Živanović and P. Vuletić, "Kernel-level DDoS protection using eBPF in IoT networks," in *Proc. 33rd Telecommunications Forum (TELFOR)*, 2025.

[36] J. Kim, "Protocol-aware and adaptive DDoS defense framework," in *Proc. IEEE 11th International Conference on Network Softwarization (NetSoft)*, 2025, pp. 257–262.

[37] M. Sakuraba, J. Kawasaki, T. Miyasaka, and A. Tagami, "An anomaly detection approach by AI/ML in IP networks with eBPF-based observability," in *Proc. Asia-Pacific Network Operations and Management Symposium (APNOMS)*, 2023, pp. 171–174.

[38] J. Rose, M. M. Cook, F. Holik, and D. Pezaros, "We're eBPF'd: Exploring adversarial manipulation of ELF files in eBPF-based programmable network stacks," in *Proc. IEEE Conference on Communications and Network Security (CNS)*, 2025.

[39] S. Zehra, H. J. Syed, and U. Faseeha, "FedMon: Federated eBPF monitoring for distributed anomaly detection in multi-cluster cloud environments," in *Proc. IEEE 20th International Conference on Emerging Technologies (ICET)*, 2025.

[40] N. S. Cherukupalle, "Zero trust enforcement through AI-driven isolation in multi-tenant Kubernetes clusters on VxRail and AWS-EKS," in *Proc. IEEE 9th International Conference on Inventive Systems and Control (ICISC)*, 2025, pp. 574–580.

[41] D. Xu and L. Chen, "A dynamic memory leak detection method for linux based on eBPF uprobe and LKM," in *Proc. 4th International Conference on Computer Systems (ICCS)*, 2024, pp. 192–197.

[42] Y. Ma, X. Shu, and R. Xie, "Memory anomaly detection and analysis based on eBPF and isolation forest," in *Proc. 44th Chinese Control Conference (CCC)*, 2025, pp. 5162–5167.

[43] N. Orzechowski, M. Zuppelli, L. Caviglione, and W. Mazurczyk, "Cryptojacking detection using eBPF and machine learning techniques," in *Proc. IEEE 10th International Conference on Information and Network Technologies (ICINT)*, 2025, pp. 22–27.

[44] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs," *IEEE Signal Processing Magazine*, vol. 30, no. 3, pp. 83–98, 2013.

[45] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed. Cambridge University Press, 2012.

[46] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.

[47] S. McCanne and V. Jacobson, "The BSD packet filter: A new architecture for user-level packet capture," in *Proc. USENIX Winter Conference*, 1993, pp. 259–269.

[48] S. Ghavamnia, T. Palit, A. Benameur, and M. Polychronakis, "Confine: Automated system call policy generation for container attack surface reduction," in *Proc. 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 443–458.

[49] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A measurement study on Linux container security," in *Proc. 34th Annual Computer Security Applications Conference (ACSAC)*, 2018, pp. 418–429.

[50] MITRE, "CVE-2021-44228: Apache Log4j2 JNDI remote code execution," https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-44228, 2021.

[51] OWASP Foundation, "OWASP top 10: 2021," Open Web Application Security Project, Tech. Rep., 2021.